VOICE-FIRST

Backend Development Master Prompt Pack

Host a Call - Screenshot-Aligned Production Backend for Mobile App and Admin Website

NestJS + PostgreSQL + Redis + Agora + Razorpay + FCM + Azure

68 sequential Codex prompts + 151-screen backend traceability

Covers setup through production sign-off: architecture, database, OTP and account gates, safe deep links, voice verification, discovery and Walkie-Talkie, real-time calls, exact 180-second video eligibility, 30-second reconnect, promotional wallet packs, billing, earnings, ranks, payout methods, referrals, notifications, safety/support, admin APIs, testing, security, deployment, and monitoring.

Screenshot-aligned revision - 18 August 2026

Prepared for Hemanth's two-person product team and Codex-assisted implementation

Read this before using the prompts

Recommended architecture: Build one modular NestJS API and one worker first. This is the fastest safe path for a two-person team. Do not begin with microservices or Kubernetes.

This pack turns the completed customer/host mobile frontend and six-module admin website into a production backend implementation sequence. Each prompt is designed to be pasted into Codex against the same repository and executed in order. The prompts require real code, migrations, automated tests, documentation, and verification - not advice-only responses.

The pack deliberately corrects requirements that changed after the frontend document was written. It also treats live money, call timing, video consent, host payouts, voice recordings, safety reports, and admin power as backend-owned responsibilities.

Legal and provider gate: This is an engineering specification, not legal, tax, payments, or compliance advice. Before live launch in India, obtain professional review of wallet/PPI characterization, host payouts and KYC, taxes, privacy notices, audio/video handling, grievance process, refunds, and intermediary obligations.

How to execute with Codex

- Use one repository and one branch per prompt or tightly related prompt group. Give Codex the repository, this prompt, and the previous prompt's completion report.
- Run prompts in numerical order. Do not skip Prompts 1-12, 26-36, 51, or 57-68; they establish invariants that later features depend on.
- Commit only after acceptance criteria and checks pass. Tag milestone releases after Foundation, Identity/Discovery, Calls/Money, Host/Admin, and Production Quality.
- Use local fake providers for ordinary development. Enable sandbox providers only in isolated preview/staging; never use live payment/payout credentials for automated tests.
- When Codex identifies a conflict between the finished frontend and this updated backend pack, preserve the visual design but update adapters/routes/contracts to the locked rules in this document.

Locked product rules

- Every call starts as voice. There is no direct-video API or signaling event.
- Video becomes eligible only after 180 server-calculated seconds during which both participants are connected to voice. Ringing and disconnected time do not count.
- Video requires an explicit customer request and an explicit host acceptance. Video billing begins only after the server has evidence that both sides entered the video media state.

- Customer wallet credits, host pending earnings, host available earnings, platform revenue, refunds, and payout clearing are separate ledger accounts.
- A host has a one-time approved self-introduction audio and at most one published daily audio status per India calendar day. Home returns the current approved status when available; otherwise it returns the approved self-introduction.
- The daily audio status is not the same as the self-introduction and is not the same as 'What's on your mind?'.
- 'What's on your mind?' is text for both customers and hosts. It is moderated, converted to curated tags, and used as one explainable matching signal; raw text is not exposed to unrelated users unless the product explicitly enables it.
- Host application verification uses a random-phrase voice challenge and manual/risk review instead of selfie verification. This is a platform trust check, not a substitute for legally required payout-provider KYC.
- No gifts or coins exist. Customer money is shown in rupees; hosts earn rupee-denominated amounts.
- Phone numbers and outside-contact details are never exposed between participants. Blocking and reporting are enforced server-side.
- The admin website has exactly six primary modules: Dashboard, Hosts, Payments, Refunds, Reports, and Settings.
- The mobile baseline is the 151-screen capture dated 18 August 2026. Production-facing screens are contracts; internal developer scenarios, component/token galleries, layout QA, motion lab, and animation lab are non-production fixtures only.
- Walkie-Talkie is a voice-only one-to-one matching shortcut. It is not a room, group call, broadcast, or direct-video path.
- The reconnect recovery window is 30 seconds for the captured v1 flow. Billing and 180-second video eligibility pause whenever both participants are not connected, and a timed-out call cannot be restored by a stale client.
- Add-money pack bonuses are promotional rupee wallet credits backed by separate ledger accounts and campaign snapshots; they are not coins, withdrawable cash, or client-calculated cashback.
- Host payout setup supports at most two saved bank accounts plus one fast-withdrawal UPI ID in v1. Sensitive values are provider-tokenized/encrypted and returned only as masked projections.
- The host Recents 'Notify user' action creates a rate-limited, preference-aware callback invitation notification. It is not chat, direct messaging, or permission to expose contact details.
- Motion, vibration, layout, Lottie/Rive/Skia, and other device presentation preferences remain client-owned unless a future explicit cross-device sync requirement is approved.

Technology baseline

Area	Selected technology	Primary responsibility
Runtime/framework	Current supported Node.js LTS + NestJS + TypeScript	REST, Socket.IO, modules, dependency injection
Architecture	Modular monolith + separate worker	Fast v1 delivery; explicit future extraction boundaries
Database	PostgreSQL + Prisma ORM with reviewed SQL for locks/ledger	Durable truth, constraints, migrations, transactions
Ephemeral/queues	Redis + BullMQ	Presence, cache, throttles, jobs, Socket.IO coordination
Contracts	OpenAPI + generated TypeScript client + versioned event schemas	Frontend integration and compatibility gates
Calls	Agora RTC with server-side AccessToken2 and signed event notifications	Voice/video transport; backend owns authorization and billing state
Payments	Razorpay adapter + signed raw-body webhooks	Order creation, verification, refund/reconciliation
Payouts	Provider-neutral payout/beneficiary adapter	Host KYC/bank verification and transfers
Media	Azure Blob-compatible object storage + scan/transcode/moderation ports	Voice verification, self-intro, daily status, attachments
Notifications	Firebase Admin/FCM with APNs configuration	Incoming call and transactional push
Observability	Pino, OpenTelemetry, Sentry, Azure Monitor/App Insights	Redacted logs, metrics, traces, alerts
Testing	Jest/Vitest-compatible tooling, Testcontainers, Supertest, Socket.IO client, k6	Unit, database, contract, E2E, concurrency, load
Delivery	Docker, Azure DevOps, Container Apps, Bicep/Terraform	Immutable images, managed secrets, staged deployment

Architecture at a glance

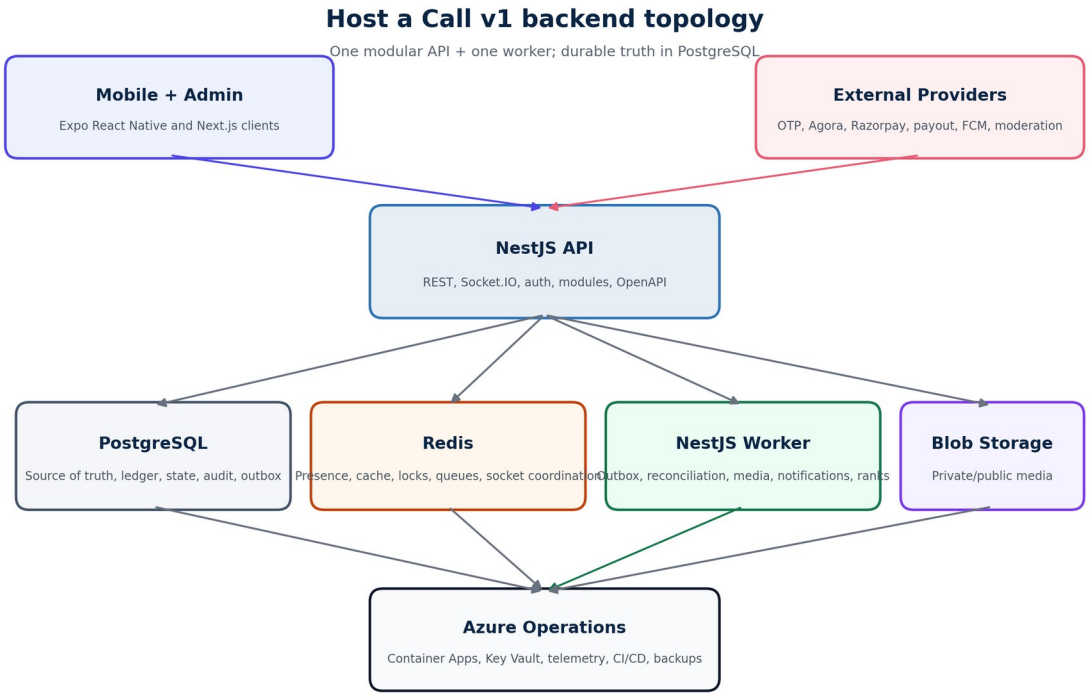


Figure 1. Recommended v1 topology. PostgreSQL remains authoritative; Redis and providers are replaceable infrastructure.

Prompt map and priority

Section	Prompts	Priority	Purpose
Foundation and architecture	1-12	P0	Required before feature implementation
Identity, profiles, content, and discovery	13-25	P0	Includes all changed requirements
Calls, billing, payments, and safety	26-43	P0	Live-money and real-time core
Host earnings, growth, and operations	44-50	P0/P1	Monetization and supply operations
Admin APIs and governance	51-56	P0	Exactly six admin modules
Production quality, deployment, and release	57-68	P0	Hardening and launch evidence

Changed requirement crosswalk

Area	Older frontend assumption	Backend pack rule
UI baseline	Earlier prompt pack inferred screens	151 screenshots and UI dumps captured 18 Aug 2026 are traced individually in Appendix L

Area	Older frontend assumption	Backend pack rule
Account gates	Mostly route/UI state	Backend bootstrap controls underage, blocked, suspended, update-required, policy and deletion outcomes
Deep links	Local route restoration	Allowlisted server resolver for referral, ticket, payment, notification and call invitation targets
Discovery	Host list and local filters	Server facets plus Featured Experts and voice-only one-to-one Walkie-Talkie matching
Host verification	Historic identity document/selfie assumption	Random-phrase voice challenge + review; payout KYC remains separate
Wallet packs	Client displays percentage extras	Campaign-snapshotted purchased and promotional rupee credits in separate ledger accounts
Calls	Frontend timers/local mock state	Server call state, exact 180,000 ms video gate, mutual consent, 30-second reconnect, verified summaries
Host payouts	UI stores UPI and two bank rows	One UPI + up to two provider-tokenized banks, one primary, review/cooling/withdrawal states
Referrals	Customer/host progress cards	Separate programs, milestone timestamps, fraud holds, ledger release and authorized detail states
Internal tools	Developer galleries are routable in dev	Deterministic fixture contracts only; disabled/not-found in production

Foundation and architecture

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were implemented, tested, documented, and committed.

Prompt 01 - Repository audit and frontend contract map

Priority: P0

Outcome: Audit the completed frontend repository and produce the exact backend integration plan without deleting working UI code.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.

- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Audit the completed frontend repository and produce the exact backend integration plan without deleting working UI code.

IMPLEMENTATION REQUIREMENTS

1. Inventory workspaces, frontend adapters, domain schemas, all 151 captured screen states, mock scenarios, deep links, environment variables, and admin module expectations.
2. Map every Auth, Customer, Host, Call, Payment, Wallet, Referral, Earnings, Withdrawal, Refund, Report, Support, Notification, and Analytics service method to an intended HTTP endpoint, WebSocket event, background job, or provider callback.
3. Create docs/backend-audit.md, docs/frontend-backend-contract-map.md, docs/ui-backend-traceability.md, docs/backend-risk-register.md, and a feature matrix marking implemented, mocked, missing, changed, client-only, or non-production.
4. Identify and document outdated or incomplete frontend assumptions: selfie verification, absence of self-intro/daily-status APIs, absence of mind-text/tag matching, locally authoritative money/call state, duplicated onboarding captures, guarded host-onboarding placeholders, empty call summaries, missing call context, and internal-only scenario data.
5. Record current build failures without broad rewrites. Add only safe scripts needed to run inspection and tests.

Acceptance criteria:

- Every production-facing captured state and every customer, host, call, money, safety, notification, and six-module admin flow has an owning backend capability or an explicitly client-owned classification.
- Changed product rules are visible as migration/integration tasks rather than hidden assumptions.
- Existing frontend checks still pass or pre-existing failures are evidenced.

END PROMPT

Prompt 02 - Architecture decision and bounded modules

Priority: P0

Outcome: Adopt a two-person-team-friendly modular monolith that remains safe to scale.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.

- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Adopt a two-person-team-friendly modular monolith that remains safe to scale.

IMPLEMENTATION REQUIREMENTS

1. Create ADRs for modular monolith, PostgreSQL source of truth, Redis ephemeral use, transactional outbox, provider adapters, REST plus Socket.IO, separate worker, and Docker-first deployment.
2. Define module boundaries: identity, customers, hosts, profiles, media, matching, presence, calls, billing, wallet, payments, earnings, ranks, withdrawals, referrals, refunds, reports, support, notifications, admin, configuration, audit, and analytics.
3. Define allowed dependency direction. Financial and call lifecycle invariants live in domain/application services, never controllers or Socket.IO handlers.
4. Document extraction triggers for future services using measured thresholds such as deploy coupling, independent scaling, database contention, or ownership - not hypothetical scale.
5. Create architecture diagrams and a request/event flow for login, top-up, call, video upgrade, refund, and withdrawal.

Acceptance criteria:

- There is one deployable API, one deployable worker, one PostgreSQL database, and one Redis dependency for v1.
- Circular module dependencies are rejected by tooling/tests.
- The ADR explains why microservices are deferred and how extraction can occur later.

END PROMPT

Prompt 03 - Backend workspace bootstrap

Priority: P0

Outcome: Add production-ready API and worker workspaces to the existing pnpm monorepo.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Add production-ready API and worker workspaces to the existing pnpm monorepo.

IMPLEMENTATION REQUIREMENTS

1. Create apps/api for NestJS HTTP/WebSocket handling and apps/worker for BullMQ consumers and scheduled reconciliation.
2. Create packages/config, packages/database, packages/contracts, packages/observability, packages/testing, and generated packages/api-client where these do not duplicate existing shared packages.

3. Use the current Node.js LTS supported by chosen dependencies, strict tsconfig, deterministic lockfile, absolute imports, dependency boundaries, and root scripts.
4. Add Dockerfiles, compose services for PostgreSQL/Redis/Azurite or compatible object storage, health/readiness endpoints, graceful shutdown, and local seed commands.
5. Keep mobile, admin, API, and worker independently buildable; do not let backend code import platform UI code.

Acceptance criteria:

- A clean checkout can install, start infrastructure, migrate, seed, run API and worker, and execute checks from documented root commands.
- Health endpoints distinguish process liveness from database/Redis/provider readiness.
- No real secret is committed.

END PROMPT

Prompt 04 - Configuration, secrets, environments, and feature flags

Priority: P0

Outcome: Create fail-fast runtime configuration for local, test, preview, staging, and production.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.

- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Create fail-fast runtime configuration for local, test, preview, staging, and production.

IMPLEMENTATION REQUIREMENTS

1. Validate environment values at startup and generate documented .env.example files for API and worker.
2. Separate public identifiers from secrets; keep OTP credentials, JWT keys, Agora certificate, Razorpay secret/webhook secret, payout credentials, FCM service identity, storage credentials, and encryption keys server-only.
3. Support provider modes fake, sandbox, and live with an explicit production denylist that prevents fake adapters in production.
4. Create backend-owned feature flags for video upgrade, audio status, mind matching, referrals, withdrawals, automatic refunds, automated moderation, and host auto-approval.
5. Serve an authenticated/public app-bootstrap projection with minimum/blocked build versions, platform store URL, maintenance status, legal-policy versions, supported languages, feature availability, and safe cache/version metadata; never let the client force its own update requirement.
6. Integrate a secret-store boundary compatible with Azure Key Vault and rotation without process-wide plaintext logging.

Acceptance criteria:

- Production boot fails when required secrets, issuer/audience values, origins, or provider modes are unsafe.
- Secrets are redacted from config dumps, errors, and diagnostics.

- Feature changes are audit-recorded and cannot silently violate locked product rules.

END PROMPT

Prompt 05 - API contracts and compatibility policy

Priority: P0

Outcome: Define stable typed REST, WebSocket, webhook, and asynchronous-event contracts that the completed frontend can integrate against.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.

- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Define stable typed REST, WebSocket, webhook, and asynchronous-event contracts that the completed frontend can integrate against.

IMPLEMENTATION REQUIREMENTS

1. Use /v1 versioning, OpenAPI, consistent pagination cursors, RFC-style problem responses, request IDs, India currency/time formatting boundaries, and UTC storage.
2. Define DTOs for every frontend adapter and use generated clients rather than hand-maintained duplicate interfaces where practical.
3. Specify idempotency-key behavior, ETag/version fields for editable resources, optimistic concurrency, and backwards-compatible additive change rules.
4. Version WebSocket envelopes with eventId, eventType, occurredAt, aggregateId, aggregateVersion, correlationId, and payload.
5. Create a deprecation policy and contract-diff CI gate. Do not expose database models directly.
6. Define a server-side deep-link resolver/allowlist for referral, support ticket, payment result, notification destination, call invitation, legal, and invalid/expired links with login restoration and resource authorization at open time.

Acceptance criteria:

- OpenAPI generation is deterministic and linted.
- Frontend adapter fixtures validate against real response schemas.
- Breaking changes fail CI unless a new API/event version and migration note are added.

END PROMPT

Prompt 06 - Canonical domain model and state machines

Priority: P0

Outcome: Implement backend-owned aggregates, invariants, enums, and transition tables before feature controllers.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not

paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement backend-owned aggregates, invariants, enums, and transition tables before feature controllers.

IMPLEMENTATION REQUIREMENTS

1. Model Account, CustomerProfile, HostProfile, HostApplication, VoiceVerification, SelfIntroduction, AudioStatus, MindPost, Tag, Availability, Call, CallParticipant, MediaSegment, PricingSnapshot, LedgerTransaction, Payment, Earning, RankSnapshot, Withdrawal, Referral, Refund, Report, SupportTicket, Notification, ConfigurationVersion, and AuditEvent.
2. Implement state machines for account, host application, voice verification, call intent, live call/media, payment, refund, earning settlement, withdrawal, referral qualification, report, and account deletion.
3. Reject invalid transitions with typed domain errors and audit security-sensitive attempts.
4. Use immutable pricing, commission, rank, configuration, and participant snapshots on financial/call records.
5. Publish domain events only after the database transaction commits through the outbox.

Acceptance criteria:

- Every state change is performed by a named command and tested transition.
- Invalid and duplicate events cannot silently advance state.
- Historical call and money records are unaffected by later profile/configuration edits.

END PROMPT

Prompt 07 - PostgreSQL schema, constraints, and migrations

Priority: P0

Outcome: Create the durable schema with database-level safeguards for identity, calls, money, safety, and administration.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.

- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Create the durable schema with database-level safeguards for identity, calls, money, safety, and administration.

IMPLEMENTATION REQUIREMENTS

1. Use UUIDv7 or another sortable non-guessable ID strategy and timestamptz for persisted time.
2. Create normalized tables, foreign keys, check constraints, unique partial indexes, and indexes based on actual access paths.
3. Enforce one active refresh-token family per device policy, one published audio status per host per India date, one rating per connected call/customer, unique provider event IDs, unique referral attribution, and balanced ledger transactions.
4. Use numeric/integer paise fields, never binary floating point; keep legal/private host data separate from public profile tables/views.

5. Implement expand-contract migration rules, migration checksums, seed separation, rollback/forward-fix guidance, and production migration locking.

Acceptance criteria:

- A blank database migrates and seeds; a production-like previous schema upgrades without data loss.
- Constraint tests prove duplicate credits, double ratings, repeated daily status, and invalid money rows are rejected.
- Query plans for key lists and reconciliations are documented.

END PROMPT

Prompt 08 - Redis, distributed coordination, presence, and cache

Priority: P0

Outcome: Use Redis for ephemeral scale while keeping PostgreSQL authoritative.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.

- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Use Redis for ephemeral scale while keeping PostgreSQL authoritative.

IMPLEMENTATION REQUIREMENTS

1. Implement namespaced keys with environment prefix, TTL ownership, serialization version, and key inventory documentation.
2. Use Redis for OTP throttles, short-lived idempotency acceleration, host presence heartbeat, call signaling rooms, rate limits, BullMQ, and carefully selected caches.
3. Implement presence as online/offline/busy/lastSeen with heartbeat expiry and database fallback; active call state remains durable in PostgreSQL.
4. Use a distributed lock only around genuinely cross-process ephemeral operations; use PostgreSQL row locks/serializable transactions for money correctness.
5. Define safe degraded behavior when Redis is slow, unavailable, or evicts data.

Acceptance criteria:

- Redis flush or restart cannot lose ledger, payment, call, application, report, or payout truth.
- Presence expires automatically and stale hosts are not presented as callable.
- Cache invalidation and maximum staleness are tested.

END PROMPT

Prompt 09 - Transactional outbox, queues, and scheduled jobs

Priority: P0

Outcome: Add reliable asynchronous processing without dual-write failures.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Add reliable asynchronous processing without dual-write failures.

IMPLEMENTATION REQUIREMENTS

1. Create outbox and inbox/deduplication tables and a publisher that claims rows safely across worker replicas.
2. Configure BullMQ queues for notifications, media processing, moderation, tagging, payment reconciliation, call reconciliation, ledger settlement, rank calculation, referral qualification, payout processing, exports, retention, and cleanup.
3. Make jobs atomic, idempotent, retryable with bounded exponential backoff and jitter, and classify retryable versus terminal errors.
4. Add dead-letter visibility, replay tooling with permission/audit controls, job timeouts, cancellation, concurrency limits, and provider rate limits.
5. Ensure an event is never marked delivered before its side effect and idempotency record are safely persisted.

Acceptance criteria:

- Crash-after-provider-success and crash-before-ack scenarios do not duplicate money or notifications.
- Failed jobs are observable and replayable without editing Redis manually.
- Queue lag and retry/dead-letter metrics exist.

END PROMPT

Prompt 10 - Provider ports, fakes, sandboxes, and circuit breakers

Priority: P0

Outcome: Isolate every external vendor behind a typed adapter and a provider-independent domain contract.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.

- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Isolate every external vendor behind a typed adapter and a provider-independent domain contract.

IMPLEMENTATION REQUIREMENTS

1. Define ports for OTP, RTC/token/notifications, payment gateway, payout, object storage, malware scanning, audio transcoding, speech-to-text, moderation, tagging, push notifications, email/SMS, and observability.
2. Implement deterministic fakes that support success, validation failure, timeout, duplicate callback, out-of-order event, partial outage, rate limit, and recovery scenarios.
3. Wrap live clients with timeouts, bounded retries, circuit breakers, bulkheads, correlation IDs, and redacted structured logging.
4. Persist provider request/reference metadata needed for reconciliation without storing prohibited card data or raw secrets.

5. Create sandbox smoke tests that are opt-in and cannot run against live credentials from ordinary CI.

Acceptance criteria:

- Feature modules depend on ports, not vendor SDK classes.
- Provider switching is configuration plus adapter registration, not controller rewrites.
- All fakes produce contract-valid data and deterministic callbacks.

END PROMPT

Prompt 11 - Authentication, authorization, and request protection foundation

Priority: P0

Outcome: Establish secure identity boundaries for customers, hosts, admins, providers, and internal jobs.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.

- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Establish secure identity boundaries for customers, hosts, admins, providers, and internal jobs.

IMPLEMENTATION REQUIREMENTS

1. Use short-lived access tokens, rotating hashed refresh tokens with reuse detection, device/session records, logout-one/logout-all, and account-status checks on every sensitive command.
2. Separate mobile user authentication, admin authentication with MFA, provider webhook authentication, and internal worker identity.
3. Implement role and permission guards plus resource ownership checks; admin UI permissions are never trusted.
4. Add CORS allowlists, secure headers, body limits, request timeout, validation whitelist, rate-limit tiers, and safe proxy/IP handling.
5. Create a permission matrix covering customer, approved host, pending host, support agent, finance operator, safety reviewer, host reviewer, and super admin.

Acceptance criteria:

- Revoked/suspended/banned sessions lose sensitive access promptly.
- Horizontal privilege escalation tests fail safely.
- Provider/internal endpoints cannot be called with ordinary user tokens.

END PROMPT

Prompt 12 - Structured logs, metrics, traces, and developer diagnostics

Priority: P0

Outcome: Make production behavior diagnosable without exposing private user data.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not

paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Make production behavior diagnosable without exposing private user data.

IMPLEMENTATION REQUIREMENTS

1. Add Pino-compatible structured logging with request/correlation IDs, event IDs, aggregate IDs, environment/release, and strict redaction.
2. Instrument OpenTelemetry traces across HTTP, WebSocket commands, database, Redis, queues, and external provider calls; integrate Sentry for exceptions.
3. Define RED/USE metrics plus product-safe metrics for OTP, host availability, call setup, connected minutes, video eligibility, billing finalization, payments, withdrawals, reports, and queue lag.
4. Create non-production diagnostic routes and scenario controls protected from production builds.
5. Write dashboards/alert stubs and a local trace/log viewing guide.

Acceptance criteria:

- A request can be followed into queued work and provider callbacks by correlation ID.
- PII redaction tests cover nested payloads and error objects.
- Health, metrics, and diagnostics cannot expose secrets or full user records.

END PROMPT

Identity, profiles, content, and discovery

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were implemented, tested, documented, and committed.

Prompt 13 - OTP login, sessions, devices, and account states

Priority: P0

Outcome: Implement production OTP authentication for customers and hosts with abuse controls and resumable onboarding.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.

- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement production OTP authentication for customers and hosts with abuse controls and resumable onboarding.

IMPLEMENTATION REQUIREMENTS

1. Create OTP request/verify endpoints with normalized E.164 phone numbers, hashed OTP storage or provider verification, purpose binding, short expiry, attempt limits, resend cooldown, daily quotas, and generic responses that resist account enumeration.
2. Support development fixed OTP only in explicitly non-production environments.
3. Issue rotating sessions after verification; bind refresh tokens to device records and record push-token association separately.
4. Route new/existing customer, new/existing host, underage, suspended, blocked, deletion-pending, mandatory-app-update, and mandatory-policy-reacceptance outcomes with a safe post-login continuation target.
5. If the current mobile adapter offers Google sign-in, verify Google OIDC tokens server-side, bind them through an explicit account-linking policy, and prevent email/phone enumeration or accidental duplicate accounts; otherwise remove the dead provider action instead of faking success.
6. Capture optional referral attribution once before account completion and prevent retroactive changes.

Acceptance criteria:

- Duplicate OTP requests and brute-force attempts are throttled by phone, device, IP/risk key, and account.
- OTP and refresh secrets are never logged or stored plaintext.
- Tests cover expiry, reuse, resend, race verification, refresh rotation/reuse, suspension, and enumeration resistance.

END PROMPT

Prompt 14 - Customer onboarding, profile, languages, and consent

Priority: P0

Outcome: Persist the complete customer onboarding flow and privacy choices used by the frontend.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.

- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Persist the complete customer onboarding flow and privacy choices used by the frontend.

IMPLEMENTATION REQUIREMENTS

1. Implement draft and completion endpoints for display name, date-only DOB, gender where used, selected approved avatar-catalog ID, primary/secondary languages, notification preference, and accepted policy versions.
2. Enforce 18+ on the server using date-only logic and an injected clock; reject inconsistent age-gate and DOB data.
3. Separate profile visibility from legal/account data and never return phone number to hosts.
4. Support resume-at-step, optimistic concurrency/version, server-driven onboarding nextStep, approved avatar catalog lookup, optional moderated upload only where the frontend truly supports it, and partial draft cleanup.
5. Lock DOB and gender after completed account setup as shown by the current profile UI; corrections require a protected support workflow and audit rather than an ordinary profile PATCH.
6. Record consent/policy evidence with version, locale, timestamp, device/session, and source screen.

Acceptance criteria:

- Interrupted onboarding resumes deterministically.
- Underage accounts cannot bypass checks through direct API calls.
- Public customer summaries contain only approved fields.

END PROMPT

Prompt 15 - Host signup, private identity, public profile, and draft lifecycle

Priority: P0

Outcome: Implement host onboarding while keeping private application data separate from the public host card.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or

provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement host onboarding while keeping private application data separate from the public host card.

IMPLEMENTATION REQUIREMENTS

1. Persist legal name, DOB, city/state, primary/secondary languages, public display name, approved avatar/photo reference, agreement versions, payout-setup status, safety-readiness status, and application draft steps for profile, photo, rules, verification, payout, agreement, and application-status routes.
2. Enforce 18+, unique application rules, required policy acknowledgement, and exact public/private projections.
3. Support draft save/resume, submit, pending review, approved, rejected with field-level corrections, resubmission, suspended, appealed, and withdrawn states.
4. Do not require selfie verification. If an avatar/photo exists, treat it as public profile content subject to moderation, not identity proof.
5. Create admin-safe projections that mask private data except for explicitly permitted review roles.

Acceptance criteria:

- Pending/rejected/suspended hosts cannot go online or receive calls.
- Public endpoints never leak legal name, phone, payout fields, verification recording, or reviewer notes.
- Resubmission changes only allowed sections and retains immutable review history.

END PROMPT

Prompt 16 - Host random-phrase voice verification

Priority: P0

Outcome: Replace selfie verification with a secure, reviewable voice challenge without pretending it is regulated KYC.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.

- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Replace selfie verification with a secure, reviewable voice challenge without pretending it is regulated KYC.

IMPLEMENTATION REQUIREMENTS

1. Create a single-use voice challenge containing a server-generated random phrase, nonce, expiry, expected language, minimum/maximum duration, and upload policy.
2. Issue a short-lived upload intent bound to host, challenge, MIME type, size, checksum, and object key; mark submissions immutable.
3. Run quality, duration, silence, duplicate-hash, replay-risk, and optional speech-to-text phrase-match checks; treat automated scores as reviewer assistance, not automatic identity truth.
4. Provide admin review states: pending processing, review required, approved, rejected, resubmission requested, expired, and risk hold, with reason codes and auditable decisions.
5. Keep raw verification audio private with a documented retention schedule. Do not create a reusable biometric voiceprint in v1.
6. Clearly separate platform voice verification from payout-provider KYC/bank verification.

Acceptance criteria:

- An old recording, expired challenge, mismatched host, reused object, or replayed request cannot be accepted.
- Approval is idempotent and required before host application approval.
- Reviewer access and playback are permissioned, audited, and time-limited.

END PROMPT

Prompt 17 - Host self-introduction audio and public playback projection

Priority: P0

Outcome: Implement the persistent 30-second host self-introduction used when no daily status exists.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.

- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement the persistent 30-second host self-introduction used when no daily status exists.

IMPLEMENTATION REQUIREMENTS

1. Create upload-intent, complete, processing, preview, submit-for-review, approve/reject, replace, and delete workflows for a maximum 30-second public self-introduction.
2. Transcode to a streaming-friendly normalized format, calculate duration/waveform metadata, scan, moderate, and store versioned media references.
3. Keep the previously approved introduction public while a replacement is pending review; do not expose rejected media.
4. Return signed short-lived playback URLs or CDN-safe references and never expose private storage keys.
5. Include audioVersion and moderation status so frontend caches invalidate correctly.

Acceptance criteria:

- Only one approved current self-introduction exists per host while history remains auditable.
- Oversize, wrong-type, too-long, malicious, or incomplete uploads fail safely.
- Playback projection contains no verification audio or internal transcript.

END PROMPT

Prompt 18 - Daily host audio status and fallback rules

Priority: P0

Outcome: Implement the separate daily audio status that temporarily replaces self-introduction on host cards.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement the separate daily audio status that temporarily replaces self-introduction on host cards.

IMPLEMENTATION REQUIREMENTS

1. Allow at most one published status per host per Asia/Kolkata calendar date; deleting/unpublishing does not restore the daily publish quota.
2. Use upload, processing, moderation, publish, active, expired, rejected, and removed states. Default expiry is 24 hours from publish, and a newly published eligible status supersedes any still-active prior status.
3. Apply the same media safety pipeline as self-introduction with its own configurable shorter duration limit.
4. Create a server projection selected in this order: active approved audio status, approved self-introduction, no audio.
5. Emit cache invalidation and notification events on publish, moderation rejection, removal, and expiry.

Acceptance criteria:

- Parallel publish attempts cannot create two daily statuses.
- Expired/rejected/private audio is never returned by Home or host detail APIs.
- Fallback behavior is deterministic across time zones and tested at India midnight boundaries.

END PROMPT

Prompt 19 - Audio processing, moderation, and outside-contact protection

Priority: P0

Outcome: Create the asynchronous safety pipeline for public host audio.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.

- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Create the asynchronous safety pipeline for public host audio.

IMPLEMENTATION REQUIREMENTS

1. Verify object checksum and metadata before processing; virus scan, decode safely, transcode, normalize loudness, calculate duration, and reject malformed media.
2. Generate speech-to-text through a provider port where enabled, then detect phone numbers, social handles, external payment/contact solicitation, sexual content, hate, threats, and prohibited advertising.
3. Use deterministic rules plus provider scores with configurable thresholds for auto-allow, manual review, and auto-block. Store reason codes, not unbounded model explanations.
4. Restrict transcript access and retention; never put raw transcript or audio in analytics/logs.
5. Support moderator override, appeals, version history, false-positive tracking, and provider outage fallback to manual review.

Acceptance criteria:

- Public media is not served before required processing/moderation completes.
- Provider retry or duplicate callback does not duplicate records or change an already final decision without an audited command.

- Contact-sharing test fixtures in Telugu, English, digits, and spoken variants are covered.

END PROMPT

Prompt 20 - What's on your mind text, moderation, and tag derivation

Priority: P0

Outcome: Implement the separate text feature for both customers and hosts and convert it into safe matching signals.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.

- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement the separate text feature for both customers and hosts and convert it into safe matching signals.

IMPLEMENTATION REQUIREMENTS

1. Create role-neutral MindPost commands for draft, publish, replace, expire, and delete with one active post per account by default and a configurable retention window.
2. Validate length/language, moderate abuse and outside-contact content, and keep raw text private by default; expose only the author's own text and approved derived tags unless product policy explicitly changes.
3. Maintain a curated tag catalog such as emotional-support, career, relationships, casual-chat, stress, study, parenting, entertainment, and language interests with locale labels and active/version status.
4. Derive tags through rules/provider adapter, include confidence and model/rule version, and let the user confirm/remove safe suggestions where the frontend supports it.
5. Exclude sensitive raw text and low-confidence tags from analytics and matching; provide a generic fallback when tagging is unavailable.

Acceptance criteria:

- Mind text is never confused with daily audio status or self-introduction in schema or API names.
- Deleted/expired posts stop affecting new match results promptly.
- Tag derivation is reproducible by version and has Telugu/English tests, moderation tests, and provider-failure tests.

END PROMPT

Prompt 21 - Host readiness, availability, presence, and busy state

Priority: P0

Outcome: Implement server-enforced host online/offline/busy behavior and readiness checks.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement server-enforced host online/offline/busy behavior and readiness checks.

IMPLEMENTATION REQUIREMENTS

1. A host may go online only when account/application/voice verification are approved, required agreements are current, required payout setup is review-ready, safety readiness is clear, no active suspension exists, and no conflicting active call exists.
2. Track requested availability durably and live presence ephemerally using authenticated heartbeats; derive callable availability from both.
3. Force offline on logout-all, ban/suspension, invalid session, prolonged heartbeat loss, or operator action; record reason and notify the host.
4. Use an atomic claim/reservation when creating a call so two customers cannot ring the same host simultaneously.
5. Return readiness blockers and masked payout warnings ordered by the profile UI's Host ready status actions without leaking internal risk scores.

Acceptance criteria:

- Stale presence, parallel call attempts, multi-device host sessions, and worker/API restarts do not create double calls.
- Pending or suspended hosts cannot bypass readiness through WebSocket events.
- Availability changes propagate to discovery within the documented SLA.

END PROMPT

Prompt 22 - Host discovery, matching, ranking, and pagination

Priority: P0

Outcome: Build explainable host discovery using language, mind tags, quality, safety, and availability without appearance-based ranking.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.

- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Build explainable host discovery using language, mind tags, quality, safety, and availability without appearance-based ranking.

IMPLEMENTATION REQUIREMENTS

1. Filter out offline/busy/suspended/blocked/conflicting hosts before scoring.
2. Score using configurable weighted components: preferred language, shared approved mind tags, rating/rank quality, repeat affinity, fair new-host exploration, availability freshness, and safety eligibility.
3. Never score on avatar/photo attractiveness, legal identity, phone, financial status, or raw private mind text.
4. Return stable cursor pagination, supported language facets, availability/quality filters, the captured All/Star/Relationship/Marriage/Confidence discovery chips as versioned curated tags, fallback-language banner metadata, data freshness, score explanation labels, public audio projection, New Host marker, city/age public fields where policy allows, and immutable price/rank display snapshots.

5. Implement Walkie-Talkie as a voice-only one-to-one match command that uses the same eligibility, blocking, language fallback, host atomic claim, pricing snapshot, and call-intent pipeline as tapping a host card; never create a room or direct-video route.
6. Record algorithm/config version and run offline evaluation fixtures for starvation, language fallback, complaint-heavy hosts, and new-host exposure.

Acceptance criteria:

- Preferred-language callable hosts appear first; safe fallback hosts appear when none exist.
- Blocked pairs never appear to either side and cannot be called by ID guessing.
- Pagination has no duplicates/skips under ordinary presence changes, with documented eventual consistency.

END PROMPT

Prompt 23 - Blocking, safety preflight, and account restrictions

Priority: P0

Outcome: Centralize pairwise safety and account eligibility checks used by discovery, signaling, and calls.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.

- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Centralize pairwise safety and account eligibility checks used by discovery, signaling, and calls.

IMPLEMENTATION REQUIREMENTS

1. Create idempotent block/unblock commands with actor, target, reason category, timestamps, and audit event; do not notify the blocked party directly.
2. Create a reusable call eligibility policy covering account state, role, age, block relation, host readiness, existing calls, self-call, minimum balance, risk holds, and geographic/product restrictions.
3. Apply the policy in Home projections, call intent creation, WebSocket signaling, token issuance, and reconnection - not only at UI preflight.
4. Support temporary safety restrictions, cooldowns, and admin force-offline with expiry and reason.
5. Return user-safe reason codes without exposing reporter identity, fraud rules, or private moderation notes.

Acceptance criteria:

- Direct endpoint or socket calls cannot bypass a block/restriction.
- Concurrent block and call creation resolve safely with a defined winner.
- Restriction changes are audited and invalidate relevant caches/sessions.

END PROMPT

Prompt 24 - Device tokens, notification preferences, and deep-link delivery

Priority: P0

Outcome: Implement reliable privacy-conscious push notifications and in-app notification history.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.

- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement reliable privacy-conscious push notifications and in-app notification history.

IMPLEMENTATION REQUIREMENTS

1. Register/rotate/revoke FCM/APNs installation identifiers per account/device and handle invalid-token feedback.
2. Store typed notification records for call, payment, refund, safety, referral, application, rank, earnings, withdrawal, support, and configuration/policy events; expose the captured Payments, Calls, Safety, Referrals, and Support preference categories.
3. Enforce user preferences except transactional/safety notices that policy requires; localize templates and validate deep-link allowlists.
4. Queue sends with collapse/deduplication keys, TTL, priority, quiet-hour policy where appropriate, delivery result, retry, and rate limits.
5. For incoming calls, generate a short-lived signed notification payload that still requires authenticated socket/API validation before accept.
6. Support cursor history, unread count, mark-one/read, mark-all-read, and authorized destination metadata; a disabled system permission is reported separately from server product preferences.

Acceptance criteria:

- No notification payload contains phone, bank/UPI, free-text report, audio transcript, OTP, or unrestricted signed URL.
- Duplicate events do not create repeated user-visible notifications.
- Mark-read, mark-all-read, pagination, invalid token cleanup, and deep-link validation are tested.

END PROMPT

Prompt 25 - Support, legal versions, privacy rights, and account lifecycle

Priority: P0

Outcome: Complete support tickets, legal content versions, data requests, logout, deactivation, and deletion workflows.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Complete support tickets, legal content versions, data requests, logout, deactivation, and deletion workflows.

IMPLEMENTATION REQUIREMENTS

1. Implement support tickets linked to call/payment/refund/withdrawal/referral/application with captured categories Account access, Notifications, Payments, Safety, and Technical; add scanned attachment upload, private notes separation, source screen/post-call metadata, assignment, status timeline, drafts, and retry-safe messages.
2. Serve versioned Terms, Privacy Policy, Community Guidelines, Host Agreement, pricing, and consent notices by locale; require reacceptance when configured.
3. Implement logout-one, logout-all, FAQs, customer deletion request/cancel/execute, host deactivation, export request, correction request, and consent/preferences update.
4. Return a typed deletion preflight with blocker code, safe explanation, destination/action, and recheck token. Block for nonzero wallet balance, active calls, unresolved payments/refunds/disputes, pending withdrawals, legal holds, or required retention; offer only legally/product-approved wallet resolution paths and do not encode the current misleading 'Open add money' label as backend policy.
5. Anonymize or delete eligible data through an auditable asynchronous workflow while retaining narrowly required financial/security records.

Acceptance criteria:

- Support attachments are scanned and permissioned.
- Deletion is resumable, idempotent, and cannot unbalance ledgers or erase audit/legal records improperly.
- Data export contains only the requesting user's allowed data and uses an expiring download.

END PROMPT

Calls, billing, payments, and safety

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were implemented, tested, documented, and committed.

Prompt 26 - Pricing, commissions, limits, and immutable configuration snapshots

Priority: P0

Outcome: Create backend-owned pricing and call policy configuration with effective dating and safe rollout.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.

- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Create backend-owned pricing and call policy configuration with effective dating and safe rollout.

IMPLEMENTATION REQUIREMENTS

1. Model voice/video customer rates, host payout percentages by rank, minimum wallet for call start, low-balance threshold, ring timeout, reconnect grace, video request cooldown, maximum call duration, refund rules, and settlement holds.
2. Use integer paise-per-minute inputs and a single documented integer formula for cumulative per-second billing; never use floats.
3. Validate non-overlapping effective periods, rank percentages, maximums, and conflicts. Changes require preview, permission, reason, effective time, and audit.
4. Snapshot the applicable version/rates/commission onto call intent and preserve it through finalization unless an explicit documented rule says otherwise.
5. Provide read endpoints optimized for mobile price strips and admin preview impact.

Acceptance criteria:

- A mid-call configuration change cannot alter that call's rate or commission.
- Invalid percentages, negative rates, gaps, overlaps, and unsafe immediate changes are rejected.
- Boundary and rounding tests prove deterministic paise totals.

END PROMPT

Prompt 27 - Double-entry wallet and platform ledger

Priority: P0

Outcome: Build the financial source of truth before integrating payments or call charging.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Build the financial source of truth before integrating payments or call charging.

IMPLEMENTATION REQUIREMENTS

1. Define an immutable chart of accounts for purchased customer wallet liability, promotional wallet liability, payment clearing, cash/provider receivable, host pending payable, host available payable, payout clearing, platform revenue, referral/promotion expense, refund clearing, chargebacks, and adjustments.
2. Every ledger transaction has balanced debit/credit entries, currency INR, idempotency/business key, source type/id, effective time, created time, description/reason code, and reversal relationship.
3. Use PostgreSQL transactions and row/advisory locking or serializable isolation for balance-changing commands; expose balances as derived/safely cached projections.
4. Implement holds/reservations for active calls without pretending held money is spent; define expiry and recovery.
5. Provide admin adjustments only as compensating postings with maker/checker permissions where required - never UPDATE/DELETE ledger rows.
6. Define spend ordering, expiry, refund, reversal, and forfeiture rules for promotional rupee credits separately from purchased credits; never expose promotional value as withdrawable cash.

Acceptance criteria:

- Database constraints/tests prove every transaction balances and cannot be mutated after posting.
- Concurrent top-up, call charge, refund, and adjustment cannot create negative spendable balance or double credit.
- A full account statement can explain every displayed balance to the paise.

END PROMPT

Prompt 28 - Razorpay order creation, verification, webhooks, and wallet credit

Priority: P0

Outcome: Integrate customer add-money so wallet credit occurs exactly once after server verification.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.

- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Integrate customer add-money so wallet credit occurs exactly once after server verification.

IMPLEMENTATION REQUIREMENTS

1. Serve versioned add-money pack/campaign options (base amount, promotional extra, badges such as Popular/Value Pack, eligibility, validity, and display copy) and create server-side orders from a pack ID or validated custom amount; never accept a client-computed bonus.
2. Create orders with validated amount limits, receipt/idempotency key, account link, campaign snapshot, currency, expiry, and provider metadata. Return only safe checkout fields.
3. Verify checkout signature server-side but treat it as one signal; process raw-body signed webhooks, fetch provider state where necessary, and credit only an allowed captured/verified state.

4. Persist provider event IDs, raw-body hash, signature result, received time, processing result, order/payment references, and out-of-order event handling without logging full payload PII.
5. Post purchased credit and any eligible promotional credit as separate balanced ledger transactions in the same database transaction as final payment state; emit events through outbox.
6. Implement success, failed, cancelled, pending, timeout, reversed, disputed, duplicate, and manual-reconcile states plus status polling, transaction list/detail, retry metadata, and payment-result deep-link resolution.

Acceptance criteria:

- Repeated checkout callbacks, webhook retries, reordered events, and worker crashes create one wallet credit only.
- The client cannot choose account, credited amount, or payment success by modifying payloads.
- Raw-body signature validation and webhook-secret rotation are tested.

END PROMPT

Prompt 29 - Call preflight, intent creation, and balance reservation

Priority: P0

Outcome: Create one idempotent call intent after all server-side eligibility checks and reserve enough wallet for a safe start.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.

- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Create one idempotent call intent after all server-side eligibility checks and reserve enough wallet for a safe start.

IMPLEMENTATION REQUIREMENTS

1. Run checks in order: account allowed, pair not blocked, host callable and atomically claimable, no conflicting call, pricing available, minimum balance for configured voice duration, and risk/region limits.
2. Create a call intent with participants, opaque RTC channel identity, pricing/config/rank snapshots, ring expiry, correlation ID, and initial state in one transaction.
3. Reserve the minimum required customer balance and release it on cancel, reject, busy, no-answer, expiry, or failure without a charge.
4. Use an idempotency key from the frontend and a host claim key/constraint to prevent rapid taps and cross-customer double ringing.
5. Return typed outcomes and safe alternative-host hints without leaking host private state.

Acceptance criteria:

- Rapid repeated taps create one intent and one reservation.
- Two customers cannot hold the same host; one gets a deterministic busy/unavailable outcome.
- No unconnected outcome posts a usage charge.

END PROMPT

Prompt 30 - Authenticated Socket.IO signaling and incoming call delivery

Priority: P0

Outcome: Implement real-time call signaling that is replay-safe, horizontally scalable, and never the financial source of truth.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.

- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement real-time call signaling that is replay-safe, horizontally scalable, and never the financial source of truth.

IMPLEMENTATION REQUIREMENTS

1. Authenticate socket connection and periodically revalidate account/session status; join only account/device rooms authorized by the server.
2. Define versioned events for call.ringing, call.accepted, call.rejected, call.cancelled, call.expired, call.state, video.requested, video.accepted/rejected/cancelled, network/reconnect hints, and call.ended.
3. Every client command includes commandId, callId, expectedVersion, and payload schema; acknowledge with durable state/version and make duplicates idempotent.
4. Persist accept/reject/cancel transitions before emitting; push incoming-call notification is a fallback hint, not permission to accept.
5. Use a Redis Streams Socket.IO adapter or documented equivalent for multi-replica recovery, with sticky-session/transport policy and database state resync after reconnect.

Acceptance criteria:

- Lost, duplicated, delayed, and reordered socket messages converge on the durable call state.
- An unauthorized device cannot subscribe to or command another call.
- Ring timeout is worker/server-owned and survives API process restart.

END PROMPT

Prompt 31 - Agora RTC token issuance and channel security

Priority: P0

Outcome: Issue least-privilege short-lived RTC tokens only for valid call participants.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or

provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Issue least-privilege short-lived RTC tokens only for valid call participants.

IMPLEMENTATION REQUIREMENTS

1. Generate AccessToken2-compatible tokens server-side using a secret certificate kept outside frontend/config output.

2. Bind token to opaque channel name, stable provider UID mapped privately to participant, role/privileges, call state, and short expiry; support renewal only while call eligibility remains valid.
3. Never accept a client-selected arbitrary channel/UID or generate wildcard production tokens.
4. Create separate join/publish permissions if supported and ensure video publish permission is withheld or server-gated until mutual eligibility/acceptance policy is satisfied.
5. Validate signed Agora event notifications and store deduplicated participant/media events for reconciliation.

Acceptance criteria:

- Nonparticipant, ended-call, blocked, expired, and suspended requests cannot obtain/renew tokens.
- Token secrets and provider customer credentials never reach mobile/admin logs or bundles.
- Clock skew, expiry, renewal, duplicate webhook, and invalid signature tests pass.

END PROMPT

Prompt 32 - Authoritative call lifecycle and connected media segments

Priority: P0

Outcome: Make the backend the source of truth for ringing, connection, media mode, reconnection, and end reasons.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.

- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Make the backend the source of truth for ringing, connection, media mode, reconnection, and end reasons.

IMPLEMENTATION REQUIREMENTS

1. Implement call states intent_created, ringing, accepted, connecting, active_voice, video_request_pending, transitioning_video, active_video, reconnecting, ending, finalizing, completed, failed, and cancelled with explicit terminal reasons.
2. Ingest provider join/leave/publish/unpublish events and authenticated client telemetry as evidence; record immutable participant/media segment events with provider timestamps, receive timestamps, sequence/source, and confidence.
3. Count a connected voice segment only when both participants are present and voice is available; create video segments only after consent and media evidence.
4. Handle backgrounding, temporary disconnect, multi-device reconnect, late provider callbacks, timeout, max duration, moderation/admin termination, and balance exhaustion.
5. Reconcile event streams into a monotonic aggregate version and expose full-state resync endpoints/events.

Acceptance criteria:

- Ringing and one-sided presence never count as connected/billable time.

- Late or duplicate events cannot reopen terminal calls or create overlapping segments.
- State recovery after API/worker restart reconstructs the same result from durable evidence.

END PROMPT

Prompt 33 - Exact 180-second voice eligibility for video

Priority: P0

Outcome: Unlock video exactly once from server-calculated dual-connected voice duration.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.

- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Unlock video exactly once from server-calculated dual-connected voice duration.

IMPLEMENTATION REQUIREMENTS

1. Accumulate eligible voice milliseconds from closed/open connected segments using an injected server clock and provider evidence; exclude ringing, reconnect grace, and one-sided connection.
2. Persist progress checkpoints and emit video.eligible once when cumulative eligible duration reaches 180,000 ms; UI timers are display only.
3. Return authoritative eligibleDurationMs, remainingMs, eligibleAt, and aggregate version on call state responses.
4. Ensure returning from video to voice does not reset earned eligibility, while a new call always starts fresh.
5. Make the threshold configuration versioned but lock the v1 production default to 180 seconds unless an audited future policy changes it.

Acceptance criteria:

- Backgrounding, reconnects, clock drift, delayed events, duplicate events, and worker restart do not unlock early or more than once.
- No direct endpoint/socket command can force eligibility.
- Boundary tests cover 179,999 ms and 180,000 ms.

END PROMPT

Prompt 34 - Mutual video request, consent, cooldown, and media transition

Priority: P0

Outcome: Implement customer-requested and host-accepted video upgrade without disrupting voice.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or

provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement customer-requested and host-accepted video upgrade without disrupting voice.

IMPLEMENTATION REQUIREMENTS

1. Allow the customer to request only after server eligibility, while active voice remains connected and accounts remain allowed.

2. Create one pending request with expiry and request version; host may accept/reject, customer may cancel, and timeout returns to voice.
3. Enforce configurable rejection/timeout cooldown and prevent duplicate or concurrent requests.
4. Record explicit consent events for both parties with policy/config version and timestamp; never infer host consent from camera publication alone.
5. Transition to active_video only after consent plus provider/media evidence. On permission/media failure, remain/return to voice and voice pricing.

Acceptance criteria:

- There is no call creation or token API that starts directly in video.
- Repeated requests cannot bypass cooldown or create multiple pending consents.
- Video billing cannot begin at request or acceptance time before media evidence.

END PROMPT

Prompt 35 - Reconnection, network degradation, and deterministic recovery

Priority: P0

Outcome: Handle temporary network loss without charging disconnected time or losing the previous media mode.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.

- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Handle temporary network loss without charging disconnected time or losing the previous media mode.

IMPLEMENTATION REQUIREMENTS

1. Enter reconnecting when participant/provider evidence crosses a configured disconnect threshold; retain previous stable media mode and start a durable 30-second v1 grace deadline captured as a configuration snapshot.
2. Pause new usage accrual while dual connection is absent. If both return within grace, resume the previous valid mode; otherwise end with a specific reconnect-timeout reason.
3. Support socket reconnection with full call-state resync, missed event recovery, token renewal, and client command deduplication.
4. On video-only degradation, stop video segment accrual and safely fall back to voice when both voice participants remain connected.
5. Persist recovery decisions and expose user-safe network quality while keeping raw provider diagnostics private.

Acceptance criteria:

- Reconnect success, timeout, one-sided return, video loss, Redis outage, and API restart tests converge on correct call/billing state.
- No disconnected interval is billable or added to 180-second eligibility.
- A stale client cannot restore a call already finalized by the server.

END PROMPT

Prompt 36 - Server-side usage billing and incremental reservations

Priority: P0

Outcome: Charge voice/video usage deterministically from authoritative media segments while preventing overspend.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.

- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Charge voice/video usage deterministically from authoritative media segments while preventing overspend.

IMPLEMENTATION REQUIREMENTS

1. Use cumulative integer formulas per media mode and post only the delta between previous and newly calculated cumulative charge; document rounding and never charge more by processing in smaller ticks.
2. Reserve wallet funds in bounded chunks rather than writing a ledger entry every second; extend reservations while balance allows and warn/end before unreserved usage exceeds risk tolerance.
3. Separate provisional usage checkpoints from final immutable call charges. Paused/disconnected time accrues zero.
4. For each finalized usage delta, post customer wallet reduction, host pending earning, and platform revenue/clearing entries using the call's rate and rank commission snapshots.
5. Make tick/finalize commands idempotent by call, segment version, and cumulative duration; recover safely after delayed jobs and process crashes.

Acceptance criteria:

- Replaying every billing job produces the same balances and one final call charge.
- Mixed voice/video/voice segments, partial seconds, reconnect gaps, and end-at-zero are paise-exact.
- Concurrent top-up and billing cannot produce unauthorized negative spendable balance.

END PROMPT

Prompt 37 - Low balance, in-call add money, and automatic end

Priority: P0

Outcome: Coordinate warnings, payment return, reservation extension, and zero-balance termination during a live call.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Coordinate warnings, payment return, reservation extension, and zero-balance termination during a live call.

IMPLEMENTATION REQUIREMENTS

1. Calculate estimated remaining time from authoritative spendable balance, current rate, existing reservation, and current media mode; label it as an estimate.
2. Emit low-balance warning once per threshold crossing/version and a critical countdown when a reservation cannot be extended.
3. Create in-call top-up orders linked to the call but use the same verified payment pipeline; pending or failed checkout never increases call funds.
4. On verified credit, atomically refresh available balance/reservation and emit an updated call financial snapshot.
5. End the call with balance_exhausted when authorized/reserved funds are consumed and no verified top-up arrives within configured grace.

Acceptance criteria:

- Payment success, failure, cancellation, delayed webhook, duplicate webhook, and call-ended-before-credit are all handled.
- A late valid credit remains in the wallet even if the call already ended, but it does not reopen the call.
- Warnings do not repeat on socket reconnect without a new threshold event.

END PROMPT

Prompt 38 - Call finalization, summaries, history, and ratings

Priority: P0

Outcome: Finalize every terminal call into immutable financial and user-facing records.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.

- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Finalize every terminal call into immutable financial and user-facing records.

IMPLEMENTATION REQUIREMENTS

1. Close open participant/media segments, reconcile provider evidence, run final cumulative billing, release unused holds, and persist a finalization version/checksum.
2. Create customer and host summaries with voice/video durations, rate/commission snapshots, charges/earnings, wallet-after-call or earning projection, end reason, verification/reconciliation status, linked support ticket, and privacy-safe participant summaries.
3. Support the captured Recents grouping/filtering, online availability hints, call detail, empty/pending skeleton, finalized state, predefined rating tags, report action, and app-restart recovery without using local persistence as financial truth.

4. Support delayed finalization state and an idempotent polling endpoint; app restart must recover pending call summary.
5. Implement cursor-paginated role-specific histories and details with filters while never exposing phone, payout data, or internal safety notes.
6. Allow one 1-5 rating with predefined tags and optional bounded comment for connected calls only; moderation applies to text.

Acceptance criteria:

- Finalization reruns do not change an already verified result except through an explicit audited adjustment/reconciliation workflow.
- Unconnected calls show zero usage charge and cannot be rated.
- History pagination, role privacy, delayed finalization, and one-rating concurrency tests pass.

END PROMPT

Prompt 39 - Customer refund requests and eligibility

Priority: P0

Outcome: Implement customer-facing refund intake without promising an automatic outcome.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.

- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement customer-facing refund intake without promising an automatic outcome.

IMPLEMENTATION REQUIREMENTS

1. Accept reason code, bounded comments, linked call/payment, and optional scanned attachment metadata; assign an immutable request ID and evidence snapshot.
2. Evaluate configurable eligibility signals such as finalization status, duplicate request, request window, connected duration, provider incident, prior refund, and open safety case.
3. Return eligible_for_review, more_information_required, duplicate, ineligible_with_safe_reason, or submitted; keep fraud/internal criteria private.
4. Link the request to a support/report timeline and notify the customer on material changes.
5. Make submission idempotent and retain draft/retry semantics through frontend adapter contracts.

Acceptance criteria:

- Multiple submits for the same call/reason window do not create duplicate open cases.
- Eligibility does not itself post money or mark the refund completed.
- Attachment access is permissioned, scanned, expiring, and audited.

END PROMPT

Prompt 40 - Refund decisions, partial allocation, and provider processing

Priority: P0

Outcome: Process full/partial/rejected refunds with balanced adjustments across customer, host, platform, and provider state.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.

- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Process full/partial/rejected refunds with balanced adjustments across customer, host, platform, and provider state.

IMPLEMENTATION REQUIREMENTS

1. Require authorized admin decision, reason, amount, allocation policy, evidence, expected case version, and maker/checker rule where configured.
2. Calculate maximum refundable amount from verified payment/call ledger, prior refunds, chargebacks, and nonrefundable policy components; use integer paise.
3. Post compensating ledger entries for customer credit/refund receivable, platform reversal, and host pending/available adjustment according to settlement state; never rewrite original earnings.
4. Call the provider refund adapter idempotently and handle submitted, processing, completed, failed, reversed, and manual-reconcile states.
5. Keep customer refund status, payment detail, host earning adjustment, and admin audit timeline consistent through outbox events.

Acceptance criteria:

- Concurrent decisions and webhook retries cannot over-refund or double-adjust host earnings.
- Partial-refund allocation is explainable to the paise and original records remain immutable.
- Provider failure leaves a visible recoverable state, not a false completed state.

END PROMPT

Prompt 41 - In-call and post-call reports, evidence, and immediate safety actions

Priority: P0

Outcome: Implement idempotent reporting that remains reachable during voice/video and supports urgent protective actions.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or

provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement idempotent reporting that remains reachable during voice/video and supports urgent protective actions.

IMPLEMENTATION REQUIREMENTS

1. Accept typed categories, bounded comment, linked call, actor/subject role, client evidence metadata, and submit-versus-submit-and-end intent.
2. Snapshot relevant call/account/config/provider references and preserve an immutable investigation timeline without recording calls by default.
3. Support immediate pair block, call termination, host force-offline, temporary restriction, priority assignment, and notification through policy-controlled commands.
4. Deduplicate repeated submits by client command/report key and allow offline draft retry without duplicate cases.
5. Apply strict access control to evidence and redact reporter identity from the reported party.

Acceptance criteria:

- Submit-and-end creates one report and one terminal call transition even when retried.
- Reports can be filed after the call and are visible in role-appropriate history.
- No public/admin list projection leaks unrestricted free text or private audio/media.

END PROMPT

Prompt 42 - Payment, call, and ledger reconciliation

Priority: P0

Outcome: Continuously detect and repair mismatches without editing financial history in place.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.

- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Continuously detect and repair mismatches without editing financial history in place.

IMPLEMENTATION REQUIREMENTS

1. Build scheduled and on-demand reconciliation for payment order versus provider payment, provider capture versus wallet credit, call evidence versus billed segments, ledger postings versus projections, refunds, and payouts.
2. Classify matches, transient pending, missing callback, duplicate provider record, amount mismatch, state mismatch, orphan record, and manual-review cases.
3. Use provider fetch APIs with rate limits and cursors; persist run/checkpoint/result records and make reruns idempotent.
4. Auto-repair only explicitly safe cases through normal commands/compensating entries; route uncertain money differences to finance review.
5. Expose reconciliation metrics, aging, CSV export, and linked evidence inside the Payments admin module.

Acceptance criteria:

- A reconciliation retry cannot duplicate wallet credits, call charges, refunds, earnings, or payouts.
- Every automatic correction has an audit trail and source evidence.

- Synthetic mismatch fixtures cover all classifications and recovery paths.

END PROMPT

Prompt 43 - Fraud signals, velocity rules, and risk holds

Priority: P0

Outcome: Add explainable anti-abuse controls for OTP, payments, referrals, calls, reports, and payouts.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.

- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Add explainable anti-abuse controls for OTP, payments, referrals, calls, reports, and payouts.

IMPLEMENTATION REQUIREMENTS

1. Create a risk-signal port and internal rule engine for device/phone/IP velocity, repeated payment failures, duplicate instruments where provider permits, self-referral links, collusive short calls, unusual refund/report patterns, and payout changes.
2. Use risk bands and reason codes to allow, challenge, delay, hold, limit, or manual-review; do not build opaque permanent bans from a single model score.
3. Keep sensitive rule thresholds server-side and versioned; log decisions with redacted inputs and support appeal/reviewer override.
4. Integrate holds with payment credit release, referral reward, host earning availability, withdrawal, and account restrictions without changing underlying earned history.
5. Create false-positive review metrics and safe simulation tools.

Acceptance criteria:

- Risk holds are idempotent, time-bounded or reviewable, and cannot create negative/unbalanced money.
- Users receive safe next-step messages rather than internal fraud details.
- Rules are tested against normal family/shared-network and retry scenarios to reduce obvious false positives.

END PROMPT

Host earnings, growth, and operations

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were implemented, tested, documented, and committed.

Prompt 44 - Host earnings accrual, holds, and release

Priority: P0

Outcome: Turn finalized calls and rewards into transparent pending and available host earnings.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.

- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Turn finalized calls and rewards into transparent pending and available host earnings.

IMPLEMENTATION REQUIREMENTS

1. Use call pricing/rank commission snapshots to post host pending earnings during/final call billing, never current rank.
2. Model settlement hold/buffer windows, refund/dispute/risk holds, release schedule, referral reward, manual adjustment, tax/withholding placeholder fields, and statement lines.
3. Release eligible pending amounts through idempotent scheduled jobs and balanced transfers between host pending and available payable accounts.
4. Return earnings dashboard, daily/monthly aggregates, call detail breakdown, available/pending/held totals, and immutable statement pagination.
5. Explain every hold/adjustment using safe reason codes and linked source IDs.

Acceptance criteria:

- Rank changes do not rewrite historical earnings.
- Concurrent refund, hold, and release jobs cannot release ineligible money or apply the same adjustment twice.
- Statement totals reconcile to ledger balances exactly.

END PROMPT

Prompt 45 - Host score, ranks, leaderboard, and prospective payout

Priority: P0

Outcome: Calculate fair versioned host ranks and leaderboards without appearance-based signals.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Calculate fair versioned host ranks and leaderboards without appearance-based signals.

IMPLEMENTATION REQUIREMENTS

1. Define calculation periods and eligible metrics: Bayesian-adjusted rating, completed connected calls, acceptance, host cancellation, repeat customers, availability contribution with caps, valid complaints, and policy penalties.
2. Normalize metrics, enforce minimum sample sizes, cap gaming-prone components, document weights, and version every calculation.
3. Assign Bronze, Silver, Gold, Platinum, or Diamond plus prospective payout percentage/effective time; store immutable rank history and explanation components.
4. Build weekly/monthly/all-time and language-filtered leaderboards with stable pagination, pinned-self lookup, freshness, insufficient-data, and under-review states. Expose metric explicitly so the dashboard earnings leaderboard and standalone completed-calls leaderboard cannot be confused or silently share incompatible ordering.
5. Support audited manual adjustment/hold and recalculation preview, never direct history rewrite.

Acceptance criteria:

- Same input/config/version produces the same result.
- New-host, sparse-rating, complaint, tie, and gaming fixtures are tested.
- A new rank affects only calls whose snapshot is created after the effective time.

END PROMPT

Prompt 46 - Withdrawal requests, beneficiary verification, and payouts

Priority: P0

Outcome: Implement secure host withdrawal from available earnings through provider-confirmed payout.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.

- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement secure host withdrawal from available earnings through provider-confirmed payout.

IMPLEMENTATION REQUIREMENTS

1. Keep payout beneficiary data tokenized/encrypted and masked; support at most two saved bank accounts, exactly one primary bank account, and one fast-withdrawal UPI ID in v1. Delegate required KYC/bank/UPI verification to an approved payout provider and store only necessary status/reference metadata.
2. Implement add, verify, make-primary, update-through-replacement, remove-if-unused, and cooling-period commands for payout methods; never return full account number/UPI after submission.
3. Validate approved host, available balance, minimum, daily limit, buffer, active holds, beneficiary status, cooling period after payout-method change, and account risk.
4. State machine: draft, submitted, review, approved, processing, completed, failed, rejected, held, reversed; use expected version and idempotency keys.

5. Move funds from available payable to payout clearing on approval/submit according to policy; settle or return through balanced entries based on provider outcome.
6. Verify payout webhooks, reconcile by provider fetch, and support retry/manual review without duplicate transfer.

Acceptance criteria:

- Parallel requests cannot withdraw the same available balance.
- Frontend/provider callbacks cannot self-mark payout complete.
- Beneficiary change, failure, reversal, duplicate webhook, and worker crash tests preserve exact balances.

END PROMPT

Prompt 47 - Customer and host referral programs

Priority: P0

Outcome: Implement separate referral programs with delayed, fraud-reviewed, ledger-backed rewards.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.

- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement separate referral programs with delayed, fraud-reviewed, ledger-backed rewards.

IMPLEMENTATION REQUIREMENTS

1. Create distinct signed deep-link/code namespaces for refer-customer and refer-host; attribute once before the referred account completes onboarding.
2. Customer qualification: joined, minimum verified add-money, first qualifying paid call completed/finalized, fraud review, reward release.
3. Host qualification: application submitted, voice verification/application approved, configured valid-call count reached, fraud review, reward release.
4. State machine supports pending, progressing, qualified, held, rewarded, rejected, expired, and reversed with versioned campaign rules/snapshots.
5. Post customer rewards to promotional wallet liability and host rewards to pending/available earnings through idempotent ledger commands; never only update counters. Return milestone timestamps, safe statuses, invite/qualified/reward counters, referral detail not-found/retry states, and campaign copy required by the captured screens.

Acceptance criteria:

- Customer and host codes cannot be confused or retroactively attached.
- Self-referral, duplicate device/account, refunded qualifying payment/call, and replayed jobs are handled.
- Progress and reward history reconcile to source events and ledger entries.

END PROMPT

Prompt 48 - Host dashboard aggregates and operational readiness

Priority: P0

Outcome: Serve the host dashboard from reliable projections without expensive per-request joins.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.

- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Serve the host dashboard from reliable projections without expensive per-request joins.

IMPLEMENTATION REQUIREMENTS

1. Create projections for current availability/readiness, rank/position/payout, today's calls/earnings, rating, acceptance, alerts, pending/available/held earnings, and next-rank progress.
2. Update projections from outbox events and support safe rebuild from source tables; include calculatedAt, sourceVersion, and partial-data warnings.
3. Return readiness blockers ordered by actionability and never include private risk or reporter details.
4. Create first-call/no-earnings states and language/time-zone aware day boundaries for Asia/Kolkata display.
5. Cache only versioned projections with explicit invalidation and a database fallback.

Acceptance criteria:

- Dashboard totals reconcile to source/ledger and expose staleness.
- Projection replay is deterministic and idempotent.
- A partial metrics failure does not incorrectly change online eligibility.

END PROMPT

Prompt 49 - Rank, earnings, application, and operational notifications

Priority: P0

Outcome: Generate one-time, deep-linked host notification experiences from durable domain events.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Generate one-time, deep-linked host notification experiences from durable domain events.

IMPLEMENTATION REQUIREMENTS

1. Create events/templates for rank up/down, payout percentage effective, earnings released/held, withdrawal completed/failed, application/voice review update, safety warning, referral reward, and availability reminder.

2. Implement the host Recents 'Notify user' callback invitation as a dedicated rate-limited command linked to a prior eligible call, recipient preference, cooldown, block/restriction policy, and notification record; it must not create chat, reveal contact data, or start a call automatically.
3. Store before/after snapshots and effective dates needed by destination screens; keep language calm and non-shaming.
4. Deduplicate by event/template/user and mark one-time celebrations as seen independently from notification read state.
5. Respect preferences/quiet hours for reminders while always delivering required transactional/safety notices.
6. Validate every deep-link destination and authorization at open time.

Acceptance criteria:

- Retries do not replay celebrations or send duplicate financial notifications.
- Rank-down details expose user-actionable metrics, not private complaints/reporters.
- Seen/read/deep-link and localization tests pass.

END PROMPT

Prompt 50 - Host moderation edits, suspension, appeal, and deactivation

Priority: P0

Outcome: Complete the host account lifecycle while preserving approved public content and money obligations.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.

- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Complete the host account lifecycle while preserving approved public content and money obligations.

IMPLEMENTATION REQUIREMENTS

1. Route public profile, self-introduction, and daily status edits through moderation; keep approved version active while a replacement is pending unless safety requires removal.
2. Implement force-offline, temporary restriction, suspension, ban, appeal, reinstatement, and reason/effective/expiry fields with permission and audit controls.
3. Block deactivation during active calls, pending withdrawals, unresolved disputes/refunds, or legal holds; retain view/withdraw access to eligible earnings according to policy.
4. Coordinate session revocation, discovery removal, call prevention, payout holds, notifications, and public projection changes through domain events.
5. Never erase historical call, ledger, rank, report, or audit evidence when a host deactivates.

Acceptance criteria:

- Suspension takes effect across REST, sockets, presence, discovery, token issuance, and withdrawals.

- Appeals cannot directly undo restrictions without authorized review.
- Moderation-pending and deactivation-blocker tests cover concurrent operations.

END PROMPT

Admin APIs and governance

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were implemented, tested, documented, and committed.

Prompt 51 - Admin authentication, MFA, RBAC, audit, and dual control

Priority: P0

Outcome: Secure the six-module admin website with server-enforced permissions and immutable operations history.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.

- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Secure the six-module admin website with server-enforced permissions and immutable operations history.

IMPLEMENTATION REQUIREMENTS

1. Implement separate admin identities, password/SSO-ready login boundary, MFA challenge, short sessions, rotating refresh or secure server session, inactivity timeout, and forced logout.
2. Define roles/permissions for dashboard view, host review, safety investigation, finance view, refund decision, payout decision, configuration management, audit export, and super admin.
3. Require step-up MFA and reason for high-risk actions; support maker/checker for large refunds, payouts, manual ledger adjustments, and sensitive configuration.
4. Record before/after hashes or safe snapshots, actor, permission, reason, request/correlation ID, IP/device context, and outcome in append-only audit events.
5. Prevent admins from reviewing/approving their own restricted action when dual control applies.

Acceptance criteria:

- Frontend route guards are supplementary; direct API calls without permission fail.
- Admin session revocation and permission changes take effect promptly.
- Audit records are searchable/exportable but cannot be edited by ordinary admins.

END PROMPT

Prompt 52 - Dashboard metrics, drill-downs, and exports

Priority: P0

Outcome: Power the Dashboard module with accurate versioned aggregates and privacy-safe drill-downs.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Power the Dashboard module with accurate versioned aggregates and privacy-safe drill-downs.

IMPLEMENTATION REQUIREMENTS

1. Provide date range/comparison metrics for customers, approved/online hosts, calls, connected minutes, 3-minute reach, video requests/acceptance, payments, host payouts, platform revenue, rewards, refunds, reports, ranks, language revenue, and referral conversion.
2. Define each metric formula, event time, inclusion/exclusion, currency, time zone, freshness, and reconciliation source in a metric catalog.
3. Use projection tables/materialized views or scheduled aggregates with partial-failure reporting; do not calculate all history on each request.
4. Constrain drill-down links to the six modules and server-authorize every filtered result.
5. Generate asynchronous CSV exports with row limits, formula-injection protection, expiring links, audit record, and no excess PII.

Acceptance criteria:

- Dashboard totals reconcile to operational records within documented latency.
- Previous-period comparison and India day boundaries are deterministic.
- Exports match current filters and permissions.

END PROMPT

Prompt 53 - Hosts module operations

Priority: P0

Outcome: Implement all host/application/verification/rank/earnings/withdrawal/referral/report operations inside Hosts.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.

- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement all host/application/verification/rank/earnings/withdrawal/referral/report operations inside Hosts.

IMPLEMENTATION REQUIREMENTS

1. Create URL-filterable cursor lists and details with profile, voice verification, application, calls, earnings, withdrawals, rank, referrals, reports, and notes projections.
2. Support approve, reject, request resubmission, voice-audio review, moderation review, force offline, restrict, suspend, ban, appeal decision, hold/release earnings, rank review, and withdrawal review.
3. Every mutation requires permission, expected version, reason, idempotency key, confirmation metadata, and audit event.
4. Serve verification/audio with short-lived reviewer URLs and log access; mask legal/payout data by field-level permission.
5. Keep applications as a Hosts subarea, not a seventh primary module.

Acceptance criteria:

- Historical pricing, rank, and earnings snapshots remain immutable after admin actions.

- Parallel reviewer decisions have one winner and a clear conflict response.
- Search/filter/export results cannot leak inaccessible private fields.

END PROMPT

Prompt 54 - Payments module operations and reconciliation

Priority: P0

Outcome: Implement payment, wallet, earnings, payout, referral reward, and reconciliation APIs within Payments.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.

- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement payment, wallet, earnings, payout, referral reward, and reconciliation APIs within Payments.

IMPLEMENTATION REQUIREMENTS

1. Provide overview KPIs and tabs for customer payments, wallet ledger/projections, host earnings, payouts, referral rewards, reconciliation runs, and manual-review cases.
2. Return payment/payout timelines with provider references, verification, webhook history summaries, ledger transaction links, refund links, and risk flags without full instrument data.
3. Support authorized hold/release, payout approve/reject/retry, reward release/hold, safe provider refresh, and compensating ledger adjustment workflows.
4. Manual adjustments require balanced posting preview, reason, permission, idempotency, and dual control above threshold.
5. Implement exports and reconciliation evidence access with field-level redaction.

Acceptance criteria:

- No admin endpoint performs direct balance UPDATE or trusts a UI-computed amount.
- Status changes invalidate/update every related projection through events.
- Reconciliation and adjustment actions remain replay-safe.

END PROMPT

Prompt 55 - Refunds and Reports module operations

Priority: P0

Outcome: Implement separate refund finance workflows and safety/support investigations.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Implement separate refund finance workflows and safety/support investigations.

IMPLEMENTATION REQUIREMENTS

1. Refunds: list/detail, eligibility evidence, linked call/payment/ledger, request information, full/partial/reject, allocation preview, provider processing, failure/retry, and immutable timeline.
2. Reports: list/detail for call safety, customer/host reports, payment complaints, referral fraud, ranking abuse, support tickets, media moderation appeals, and account appeals.
3. Support priority, assignment, SLA, internal notes, evidence access, linked records, warn/restrict/suspend/ban, block, force offline, hold earnings, rank review, close/reopen, and escalation.
4. All destructive/financial actions require expected version, permission, reason, confirmation, and audit; evidence access is logged.
5. Keep support tickets as a Reports subtype and preserve Refunds and Reports as separate primary modules.

Acceptance criteria:

- Partial refund and safety action integration tests verify ledger, account, notification, and audit effects.
- Reported users cannot learn reporter identity or internal notes.
- Case history is append-only and concurrent assignments/decisions are conflict-safe.

END PROMPT

Prompt 56 - Settings, effective-dated configuration, and change governance

Priority: P0

Outcome: Make Settings the audited source for call, pricing, ranking, referral, withdrawal, language, safety, provider, notification, and admin configuration.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.

- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Make Settings the audited source for call, pricing, ranking, referral, withdrawal, language, safety, provider, notification, and admin configuration.

IMPLEMENTATION REQUIREMENTS

1. Define validated schemas with draft, preview, submit, approve where required, scheduled effective time, activate, supersede, rollback-as-new-version, and history.
2. Prevent conflicts such as video threshold below locked policy, negative rates, payout percentages over allowed range, rank weights not totaling expected value, unsafe low balance, or overlapping effective versions.
3. Provide impact previews for price/commission, rank distribution sample, referral qualification, withdrawal limits, and notification template rendering.
4. Encrypt/redact provider secret settings; expose only public identifier/status/last test time to ordinary admin responses.
5. Emit configuration-changed events and snapshot applicable versions on new calls/campaigns while leaving history untouched.

Acceptance criteria:

- Dirty UI or forged payload cannot bypass server validation/approval.
- Rollback creates a new audited version rather than deleting history.
- Settings changes are deterministic across API replicas and time-zone boundaries.

END PROMPT

Production quality, deployment, and release

Execute the prompts in this section in numerical order. Each prompt assumes all previous prompts were implemented, tested, documented, and committed.

Prompt 57 - Privacy, retention, consent, and India compliance engineering

Priority: P0

Outcome: Translate privacy and regulatory review requirements into configurable technical controls and evidence.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.

- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Translate privacy and regulatory review requirements into configurable technical controls and evidence.

IMPLEMENTATION REQUIREMENTS

1. Create a data inventory mapping purpose, lawful/contractual basis for legal review, collection point, visibility, encryption, retention, deletion/anonymization, export, processor/provider, and India-region requirement.
2. Implement consent/notice records, policy reacceptance, access/correction/export/deletion requests, breach-contact/runbook placeholders, grievance/support routing, and child exclusion through 18+ policy.
3. Set specific retention classes for OTP, sessions, verification audio, public audio/transcripts, mind text/tags, calls, provider events, reports/evidence, financial ledgers, audit logs, and operational logs.
4. Design CERT-In-aligned security logging retention/location controls and incident export while strictly excluding secrets and unnecessary content.
5. Add a legal launch gate for wallet/PPI characterization, host payout/tax/KYC, recording/monitoring, intermediary obligations, terms, refunds, and grievance process; do not claim code alone provides compliance.

Acceptance criteria:

- Retention/deletion jobs are idempotent, legal-hold aware, auditable, and tested on linked data.
- A privacy export cannot include another user's data, reporter identity, internal risk rules, or provider secrets.
- Production release checklist identifies owner and evidence for every legal-review item.

END PROMPT

Prompt 58 - Security hardening and threat model

Priority: P0

Outcome: Harden the backend against account takeover, payment fraud, call abuse, data leakage, and operator misuse.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.

- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Harden the backend against account takeover, payment fraud, call abuse, data leakage, and operator misuse.

IMPLEMENTATION REQUIREMENTS

1. Create a STRIDE-style threat model and abuse cases for OTP, refresh tokens, deep links, media upload, WebSocket commands, RTC tokens, provider webhooks, payment credit, ledger, refund, payout, admin, exports, SSRF, injection, and supply chain.
2. Implement request/body/media limits, validation, SQL-safe access, SSRF-safe provider/media fetches, egress allowlists where possible, malware scanning, signed URL scope/TTL, encryption, secret rotation, and dependency/container scanning.
3. Add webhook replay windows, raw-body signature verification, IP allowlist only as supplementary control, idempotency, and alerting on repeated failures.
4. Protect admin and sensitive evidence with MFA, least privilege, step-up, access audit, export watermark/limits where appropriate, and break-glass procedure.
5. Run SAST, secret scan, dependency audit, container scan, and targeted DAST in CI with documented exception ownership/expiry.

Acceptance criteria:

- Critical threat mitigations have executable tests or verifiable configuration evidence.
- No high/critical unresolved finding is silently waived.
- A compromise of a mobile token alone cannot credit wallet, change call billing, approve host, refund, or withdraw earnings.

END PROMPT

Prompt 59 - Unit, integration, database, and property testing

Priority: P0

Outcome: Build high-value automated coverage for domain invariants and infrastructure behavior.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Build high-value automated coverage for domain invariants and infrastructure behavior.

IMPLEMENTATION REQUIREMENTS

1. Unit-test all state machines, pricing/rounding, balance/ledger postings, eligibility, matching weights, tag policies, rank calculations, retention rules, and permission policies with injected clocks/IDs.
2. Use Testcontainers for PostgreSQL and Redis integration tests covering real constraints, transactions, locks, migrations, outbox, queues, and Socket.IO coordination.

3. Add property-based tests for balanced ledger entries, cumulative billing chunk independence, no over-refund, no over-withdrawal, state transition safety, and pagination cursor stability.
4. Create fixture builders that default to privacy-safe synthetic data; no production dump in test environments.
5. Set meaningful coverage/branch gates on critical domain/application modules rather than superficial global 100 percent.

Acceptance criteria:

- Tests fail on mutation of locked invariants and run deterministically in CI.
- Parallel/concurrency tests reproduce races for top-up, call claim, billing, audio status, rating, refund, and withdrawal.
- Migration tests cover blank and upgrade paths.

END PROMPT

Prompt 60 - Contract, E2E, WebSocket, and provider sandbox testing

Priority: P0

Outcome: Automate complete backend journeys and frontend-compatible contracts before live credentials are used.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.

- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Automate complete backend journeys and frontend-compatible contracts before live credentials are used.

IMPLEMENTATION REQUIREMENTS

1. Run OpenAPI and event-schema compatibility tests against frontend adapters/generated clients.
2. Build API/WebSocket E2E journeys for customer signup, account gates/deep links, host signup/voice verification, intro/status/mind tags, discovery/Walkie-Talkie, promotional top-up, call, 180-second video accept/reject, 30-second reconnect, billing, summary/rating, refund/report, notification preferences, support, deletion blockers, earnings/rank, payout methods/withdrawal, referrals, callback invitations, and six admin modules.
3. Simulate duplicate/out-of-order/missing payment and Agora callbacks, network partitions, worker crashes, Redis restart, and database retryable transaction errors.
4. Add opt-in sandbox tests for OTP, Razorpay, Agora token/join, object storage, FCM, and payout provider with isolated credentials/data and cleanup.
5. Publish machine-readable test results and contract diffs as CI artifacts.

Acceptance criteria:

- Critical journeys run with local fakes and do not require paid providers for ordinary CI.
- Sandbox failures cannot mutate production accounts or send real payouts/messages.
- Frontend contract fixtures and backend schemas remain synchronized.

END PROMPT

Prompt 61 - Performance, load, concurrency, and capacity budgets

Priority: P0

Outcome: Prove the modular monolith handles launch traffic and fails predictably under stress.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.

- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Prove the modular monolith handles launch traffic and fails predictably under stress.

IMPLEMENTATION REQUIREMENTS

1. Define SLO/budgets for OTP, Home discovery, call intent, socket signaling, token issuance, payment webhook, billing/finalization, admin lists, queue lag, and database utilization.
2. Create k6 or equivalent tests for read traffic, host presence heartbeats, concurrent ringing claims, active socket connections, payment webhook bursts, billing jobs, and admin exports using synthetic data.
3. Measure query plans, N+1 behavior, pool sizing, indexes, Redis memory/TTL, queue concurrency, Node event-loop lag, and worker backpressure.
4. Implement admission control, rate limits, bulkheads, export row limits, bounded pagination, timeouts, and graceful degradation for noncritical matching/audio/analytics features.
5. Document capacity per API/worker replica and scale triggers; keep financial/call correctness ahead of raw throughput.

Acceptance criteria:

- No load test creates duplicate call claims or money postings.
- Critical SLOs meet agreed launch target at expected peak plus headroom.
- Overload produces rate-limited/degraded responses, not uncontrolled memory growth or corrupted state.

END PROMPT

Prompt 62 - Database operations, backups, restore, and disaster recovery

Priority: P0

Outcome: Make production data recoverable and schema changes operationally safe.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Make production data recoverable and schema changes operationally safe.

IMPLEMENTATION REQUIREMENTS

1. Configure managed PostgreSQL backups/PITR, retention, encrypted storage, restricted network access, connection pooling, maintenance, query monitoring, and least-privilege roles for migration/API/worker/read-only operations.
2. Document RPO/RTO, backup ownership, restore drill, region choice, Redis recovery expectations, object storage versioning/lifecycle, and provider data dependencies.

3. Create pre-deploy migration checks, backups for high-risk changes, expand-contract procedure, data backfill jobs, validation queries, and forward-fix/rollback decision rules.
4. Test restore into an isolated environment, verify migration version, ledger balance, row counts/checksums, critical samples, and signed URL/provider secret isolation.
5. Create runbooks for database saturation, slow query, failed migration, accidental deletion, corrupted projection, and lost Redis.

Acceptance criteria:

- A documented restore drill succeeds within target and proves ledger/call/audit integrity.
- Projections can rebuild; source financial history cannot be silently regenerated from caches.
- Production migration never runs from every API replica at startup.

END PROMPT

Prompt 63 - Frontend integration, generated client, and migration from mocks

Priority: P0

Outcome: Connect the completed mobile/admin frontends to real APIs incrementally without screen rewrites.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.

- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Connect the completed mobile/admin frontends to real APIs incrementally without screen rewrites.

IMPLEMENTATION REQUIREMENTS

1. Generate and publish a typed API client plus WebSocket event types from versioned contracts; map backend problem codes to existing frontend error normalization.
2. Create an integration matrix for every frontend adapter method, all 151 captured screen states, query key invalidation, pagination cursor, deep link, and real-time event; classify internal galleries/scenarios as non-production fixtures.
3. Replace mocks feature-by-feature behind environment flags: auth/profile, media/mind/discovery, wallet/payment, calls, host earnings/rank/withdrawal/referral, then admin.
4. Update outdated frontend flows for voice verification, self-introduction, daily audio status fallback, and mind-text/tag matching while preserving finished design.
5. Add preview environment seed personas and deterministic provider sandbox controls; remove any client code that computes authoritative balance, billing, rank, or permission.

Acceptance criteria:

- Switching mock versus real remains configuration-driven during rollout.
- Contract tests cover all screens' loading/empty/error/pending and state-machine outcomes.
- No dead buttons or hidden fallback mocks remain in production mode.

END PROMPT

Prompt 64 - Azure DevOps CI/CD and supply-chain controls

Priority: P0

Outcome: Create protected, repeatable build-test-scan-migrate-deploy pipelines.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.

- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Create protected, repeatable build-test-scan-migrate-deploy pipelines.

IMPLEMENTATION REQUIREMENTS

1. Use frozen pnpm installs, pinned actions/tasks/images, reproducible builds, format/lint/typecheck, unit/integration/contract/E2E gates, migration validation, OpenAPI diff, SAST, secret/dependency/container scans, and SBOM/provenance artifacts.
2. Build immutable API/worker images once and promote the same digest through preview, staging, and production.
3. Use environment approvals, workload/managed identity, Key Vault references, least-privilege service connections, protected branches, and no secrets in YAML/logs.
4. Run migrations as a single controlled job before compatible app rollout; add smoke/health checks, deployment annotations, traffic shift, and automatic rollback criteria that do not reverse unsafe schema changes blindly.
5. Retain release, test, scan, migration, and deployment evidence with clear ownership.

Acceptance criteria:

- A clean commit can produce and deploy preview without manual workstation steps.
- Production requires approved gates and cannot deploy failed tests/scans/migrations.
- Rollback restores the previous image while preserving data correctness.

END PROMPT

Prompt 65 - Azure infrastructure as code and production topology

Priority: P0

Outcome: Provision a cost-aware secure Azure baseline without Kubernetes.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Provision a cost-aware secure Azure baseline without Kubernetes.

IMPLEMENTATION REQUIREMENTS

1. Use Bicep or Terraform for resource groups, Container Apps environment, API min-one replica, worker/jobs scaling, PostgreSQL Flexible Server, managed Redis-compatible service, Blob Storage, Key Vault, Container Registry, Log Analytics/Application Insights, DNS/TLS, and optional Front Door/WAF.

2. Use India-region resources where required by policy/legal review, private networking where cost/plan allows, managed identities, least-privilege RBAC, firewall rules, TLS, encryption, and separate environments/subscriptions or resource groups.
3. Configure WebSocket ingress, sticky/transport behavior required by the Socket.IO adapter, health probes, graceful termination, autoscaling, connection draining, and deployment revision strategy.
4. Create budgets, cost tags, alerts, environment scale settings, and a low-cost preview topology that never shares production data/secrets.
5. Document vendor-neutral Docker deployment so Azure services can be replaced without product-module rewrites.

Acceptance criteria:

- IaC plan is reviewable, repeatable, and contains no secret values.
- Preview/staging/production are isolated and use correct provider modes.
- Socket, database, Redis, storage, and secret access survive a smoke test after deployment.

END PROMPT

Prompt 66 - Monitoring, alerts, SLOs, and incident runbooks

Priority: P0

Outcome: Create actionable operations coverage for a two-person team.

> **Codex instruction:** Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.

- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Create actionable operations coverage for a two-person team.

IMPLEMENTATION REQUIREMENTS

1. Build dashboards for API latency/errors, socket connections/reconnects, database/Redis, queues, provider health, OTP, presence, call setup/connection/finalization, 180-second eligibility, payments, ledger/reconciliation, refunds, reports, earnings, withdrawals, and notification delivery.
2. Define SLOs and burn-rate or pragmatic threshold alerts with severity, owner, channel, silence/maintenance policy, and linked runbook; avoid alerting on every single retry.
3. Add synthetic canaries for health, login fake/sandbox, discovery, top-up status, token issuance, and admin read-only flow without creating real money/calls.
4. Write runbooks for provider outage, webhook backlog, stuck call/finalization, balance mismatch, payout failure, mass false moderation, FCM outage, database/Redis incident, secret rotation, and security incident.
5. Include safe feature-flag kill switches and manual reconciliation/replay procedures with audit.

Acceptance criteria:

- Every P0 alert has an owner, first diagnostic query/dashboard, containment step, and recovery verification.
- Runbooks never instruct operators to edit ledger/call history directly.

- A game-day exercise validates at least payment webhook backlog, Agora outage, and Redis restart.

END PROMPT

Prompt 67 - Release readiness, provider go-live, and operational documentation

Priority: P0

Outcome: Perform the full backend launch audit and create all developer/operator handoff material.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.

- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Perform the full backend launch audit and create all developer/operator handoff material.

IMPLEMENTATION REQUIREMENTS

1. Audit every REST route, socket command/event, webhook, queue/job, state transition, permission, idempotency key, database constraint, projection, feature flag, provider mode, log redaction, retention rule, test, SLO, backup, and runbook.
2. Complete provider go-live checklists for OTP/DLT, Agora project/token/webhooks, Razorpay keys/webhooks/settlement, payout beneficiary/KYC, FCM/APNs, storage/CDN, moderation/STT, Sentry/telemetry, and Azure quotas.
3. Create README, architecture, module boundaries, data model, API/events, provider integration, local development, testing, migrations, frontend integration, security/privacy, deployment, monitoring, incident response, backup/restore, and known limitations docs.
4. Run clean-checkout build, migration/seed, all tests/scans, provider sandbox tests, load smoke, restore evidence review, and staging end-to-end release rehearsal.
5. Produce BACKEND_RELEASE_REPORT.md with revision, versions, migrations, feature matrix, commands/results, performance/security/privacy findings, provider prerequisites, risks, and go/no-go.

Acceptance criteria:

- No fake provider, developer tool, sample secret, PII fixture, or admin bypass is enabled in production.
- All P0 checks are green and remaining P1/P2 limitations have owner/date/mitigation.
- A new developer can run and understand the backend from a clean checkout.

END PROMPT

Prompt 68 - Final integration, remediation, and production sign-off

Priority: P0

Outcome: Treat the complete mobile, admin, API, worker, providers, and infrastructure as one release candidate and repair every backend-owned gap.

Codex instruction: Copy this complete prompt into Codex and run it against the current repository. Do not paste only the requirements list.

You are the principal backend engineer responsible for the Host a Call production codebase. Work in the existing repository and inspect its current state before editing. Implement this stage completely; do not only explain or provide snippets. Preserve all previously completed customer, host, admin, and shared frontend flows unless this updated backend pack explicitly changes their contract.

Global engineering rules:

- Work inside the existing repository. Inspect current code, frontend service adapters, packages/domain contracts, route inventory, the 18 August 2026 screenshot manifest, and environment files before editing. Preserve completed mobile and admin work.
- Implement the stage completely; do not only explain or provide snippets. Make safe repository edits, migrations, tests, documentation, and runnable configuration.
- Use a modular monolith plus a separate worker process. Do not introduce microservices, Kafka, Kubernetes, or event sourcing unless measured scale proves they are required and an ADR is approved.
- Use strict TypeScript. Validate every external payload. Treat the mobile app, admin browser, payment callbacks, RTC callbacks, notification tokens, and uploaded media as untrusted input.
- All money is integer paise. Use immutable double-entry ledger postings, database transactions, idempotency keys, and explicit reconciliation. Never trust a client balance, client timer, payment success callback, rank score, payout status, or admin permission.
- Every mutating HTTP command and provider webhook must be idempotent. Use unique constraints and transactional state transitions, not check-then-write logic vulnerable to races.
- Use PostgreSQL as the source of truth; Redis is only for ephemeral presence, locks, throttles, cache, Socket.IO coordination, and queues. Correctness must survive Redis loss.
- Protect privacy: redact phone numbers, legal identity, bank/UPI data, free-text safety reports, audio transcripts, tokens, and provider secrets from logs, analytics, traces, and fixtures.
- Create deterministic local fakes for OTP, Agora, Razorpay, payout, object storage, moderation, tagging, and FCM. Production provider selection must be configuration-driven, not scattered conditionals.
- Add or update unit, integration, contract, and failure-path tests for this stage. Use injected clocks and deterministic IDs where time or ordering affects behavior.
- Run format, lint, typecheck, relevant tests, migration validation, and builds before finishing. Fix backend-owned failures; document only genuine external prerequisites.
- At completion report changed files, migrations, API or event contract changes, commands and results, security/privacy impact, operational metrics added, frontend integration notes, and remaining provider prerequisites.
- Keep docs/ui-backend-traceability.md current. Every production screenshot state must map to a real endpoint, event, job, provider callback, client-only behavior, or a typed unavailable/empty/error state; internal developer and gallery routes must be disabled in production.

STAGE OBJECTIVE

Treat the complete mobile, admin, API, worker, providers, and infrastructure as one release candidate and repair every backend-owned gap.

IMPLEMENTATION REQUIREMENTS

1. Compare implementation against this prompt pack, the 151-row screenshot traceability register, frontend adapter map, locked product rules, schema/API/event appendices, and release report. Do not assume earlier prompts were completed correctly.
2. Find and fix broken contracts, unhandled states, unsafe booleans, race conditions, missing constraints, stale projections, dead actions, mock leakage, unredacted data, weak permissions, unreconciled money, flaky jobs/tests, and undocumented operations.
3. Prove there is no direct video path; 180 seconds is backend-calculated; the 30-second reconnect gap is non-billable; consent is mutual; daily audio status/self-intro/mind text are distinct; voice verification replaces selfie; Walkie-Talkie is one-to-one voice only; and blocked pairs cannot connect.
4. Prove every wallet/payment/call/refund/earning/withdrawal/referral operation is integer-paise, balanced, immutable, idempotent, concurrent-safe, and reconcilable.
5. Run final clean build, database upgrade, full automated suite, contract diff, scans, load target, staging E2E, provider sandbox, backup/restore review, alert/runbook check, and frontend real-adapter smoke flows.
6. Update BACKEND_RELEASE_REPORT.md with exact evidence and a final go/no-go. Never declare production-ready with red P0 checks or unverified live-money/call prerequisites.

Acceptance criteria:

- Mobile and admin complete their critical real-backend journeys in staging, including every production screen contract and captured failure/empty/pending state.
- No broken endpoint/event, double money movement, early video unlock, safety bypass, secret, PII log, or release-blocking defect remains.
- The system is either approved with evidence or explicitly no-go with a bounded remediation list.

END PROMPT

Appendix A - Recommended repository shape

```
host-a-call-platform/
|-- apps/
|   |-- mobile/              Expo React Native customer + host app
|   |-- admin/               Next.js six-module admin
|   |-- api/                 NestJS REST + Socket.IO modular monolith
|   `-- worker/              BullMQ consumers, schedules, reconciliation
|-- packages/
|   |-- contracts/           API/event schemas and shared enums
|   |-- api-client/          Generated frontend client
|   |-- database/            Prisma schema, migrations, SQL, seeds
|   |-- config/              Runtime configuration schemas
|   |-- observability/       Logging, metrics, tracing, redaction
|   |-- testing/             Builders, containers, provider fakes
|   |-- design-tokens/       Existing frontend package
|   `-- domain/              Existing frontend-facing domain package
|-- infra/
|   |-- compose/             Local PostgreSQL, Redis, Azurite
|   `-- azure/               Bicep or Terraform by environment
|-- docs/
|-- scripts/
|-- tests/
|   |-- contract/
|   |-- e2e/
|   |-- load/
|   `-- sandbox/
|-- pnpm-workspace.yaml
`-- azure-pipelines.yml
```

Appendix B - Core data model catalog

Domain	Primary tables	Critical constraints
Bootstrap/legal	app_release_policies, maintenance_windows, legal_documents, faqs, deep_link_nonces	Platform/build gates; locale/version/effective date; allowlisted authorized destinations
Identity	accounts, account_roles, devices, sessions, refresh_tokens, otp_challenges, oauth_identities, policy_acceptances	Normalized identity; tokens hashed; session family/device linkage; status checks
Customer	customer_profiles, customer_languages, avatar_catalog, customer_avatar_selections	18+; DOB/gender locked after setup; public projection excludes phone/private fields
Host	host_profiles_public, host_profiles_private, host_applications, application_reviews, host_restrictions	Public/private split; state transitions; reviewer history

Domain	Primary tables	Critical constraints
Voice verification	voice_challenges, voice_submissions, voice_processing_results, voice_reviews	Single-use nonce; immutable object/checksum; private retention
Public audio	self_introductions, audio_statuses, media_objects, media_processing, moderation_decisions	One approved intro; one daily publish quota per India date; versioned fallback
Mind and tags	mind_posts, tag_catalog, mind_post_tags, tag_versions	One active post; moderated; derived tag confidence/version
Presence/matching	availability_preferences, presence_snapshots, match_config_versions, match_explanations	PostgreSQL preference; Redis live presence; no appearance signal
Calls	call_intents, calls, call_participants, call_events, media_segments, video_consents, call_holds, call_finalizations	Aggregate version; no overlapping segments; immutable snapshots
Pricing	pricing_versions, rank_commission_versions, call_policy_versions	Effective-dated; no overlap; snapshot on call
Ledger	ledger_accounts, ledger_transactions, ledger_entries, balance_projections, reservations	Purchased/promotional/pending/available accounts; integer paise; balanced; append-only
Payments	wallet_pack_campaigns, payment_orders, provider_payments, provider_webhook_events, payment_reconciliation	Server bonus snapshot; unique provider events; raw-body hash; one purchased/promotional credit
Earnings/payouts	host_earnings, earning_holds, withdrawal_requests, payout_beneficiaries, payout_method_changes, provider_payouts	One UPI + max two banks; one primary; available lock; provider-confirmed completion
Growth	referral_codes, referral_attributions, referral_progress, referral_rewards, campaigns	Distinct customer/host programs; one attribution; ledger-backed reward
Safety/support	blocks, reports, report_events, evidence_objects, support_tickets, ticket_messages, legal_holds	Reporter privacy; immutable timeline; scanned evidence
Refunds	refund_requests, refund_decisions, provider_refunds, refund_allocations	No over-refund; compensating ledger entries; idempotent provider command
Notifications	installations, notification_preferences, notifications, delivery_attempts, callback_invitations	Five UI categories; no PII; dedupe; deep-link allowlist; callback cooldown
Admin/governance	admins, admin_roles, permissions, admin_sessions, audit_events, configuration_versions, exports	MFA; least privilege; append-only audit; expiring exports
Reliability	outbox_events, inbox_events, job_runs, reconciliation_runs, projection_checkpoints	Transactional publish; dedupe; safe replay/checkpoints

Appendix C - REST API catalog

Method	Path	Purpose
GET	/v1/bootstrap	Build/update gate, maintenance, legal and feature metadata
POST	/v1/deep-links/resolve	Allowlisted login-restorable destination resolution
GET	/v1/legal/{document} | /v1/faqs	Public locale/versioned legal and FAQ content

Method	Path	Purpose
POST	/v1/auth/otp/request	Request purpose-bound OTP with throttling
POST	/v1/auth/otp/verify	Verify OTP and create/continue account session
POST	/v1/auth/google	Verify OIDC token and apply explicit account-linking policy
POST	/v1/auth/refresh	Rotate refresh token and issue short access token
POST	/v1/auth/logout | /logout-all	Revoke one session or every session
GET/ PATCH	/v1/customers/me	Read/update customer profile with version
GET	/v1/avatar-catalog | /v1/languages	Approved avatars and supported languages
PUT	/v1/customers/me/onboarding-draft	Save/resume onboarding step data
POST	/v1/customers/me/complete-onboarding	Validate 18+ and complete onboarding
GET/ PATCH	/v1/hosts/me	Private host profile and state
PUT/POST	/v1/hosts/me/application-draft | /submit	Save or submit host application
POST	/v1/hosts/me/voice-verification/challenge	Create random phrase challenge
POST	/v1/hosts/me/voice-verification/upload-intent	Create scoped private upload
POST	/v1/hosts/me/voice-verification/complete	Complete immutable submission
POST	/v1/hosts/me/self-introduction/upload-intent	Create public-intro upload intent
POST	/v1/hosts/me/self-introduction/submit	Process/moderate replacement
POST	/v1/hosts/me/audio-status/upload-intent	Create daily status upload intent
POST	/v1/hosts/me/audio-status/publish	Publish within India daily quota
GET/ PUT/ DELETE	/v1/me/mind	Read, publish/replace, or delete own mind text
GET	/v1/tags	Curated localized tag catalog
POST	/v1/hosts/me/availability	Request online/offline with readiness
POST	/v1/hosts/me/heartbeat	Authenticated presence heartbeat
GET	/v1/discovery/hosts | /facets	Filtered scored cursor host list and supported facets
POST	/v1/matches/walkie-talkie	Voice-only one-to-one match through normal call intent
POST/ DELETE	/v1/blocks/{accountId}	Block/unblock pair with policy checks
GET	/v1/blocks | /v1/reports	Authorized block list and safe report history
GET	/v1/wallet	Customer spendable/held balance projection
GET	/v1/wallet/transactions | /{id}	Immutable statement and transaction detail
GET	/v1/wallet/packs	Versioned base/promotional add-money pack catalog
POST	/v1/payments/orders	Create verified order from server pack/campaign snapshot

Method	Path	Purpose
GET	/v1/payments/{paymentId}	Poll server-owned payment/credit state
POST	/v1/webhooks/razorpay	Raw-body signed payment/refund webhook
POST	/v1/calls/intents	Eligibility, host claim, price snapshot, reservation
POST	/v1/calls/{id}/cancel	Cancel ringing intent idempotently
POST	/v1/calls/{id}/rtc-token	Issue participant-bound RTC token
POST	/v1/webhooks/agora	Signed participant/media events
GET	/v1/calls/{id}/state	Authoritative resync including timers/financials
POST	/v1/calls/{id}/video-request	Eligible customer request
POST	/v1/calls/{id}/video-response	Host accept/reject with version
POST	/v1/calls/{id}/end	Idempotent participant end command
GET	/v1/calls | /v1/calls/{id}	Role-specific history and detail
GET	/v1/calls/{id}/summary	Pending/finalized customer or host summary
POST	/v1/calls/{id}/callback-invitations	Rate-limited host notification to prior customer
POST	/v1/calls/{id}/rating	One rating for connected customer call
POST	/v1/calls/{id}/reports	In/post-call safety report
POST	/v1/calls/{id}/refund-requests	Customer refund intake
GET	/v1/hosts/me/earnings	Pending/available/held dashboard
GET	/v1/hosts/me/earnings/statement	Immutable host statement
GET	/v1/ranks/me | /v1/leaderboards?metric=	Rank plus explicit earnings/completed-call leaderboards
GET/ POST/ PATCH/ DELETE	/v1/hosts/me/payout-methods	Masked UPI/bank methods, verify and make primary
POST	/v1/withdrawals	Validated locked withdrawal request
GET	/v1/withdrawals | /v1/withdrawals/{id}	Withdrawal history/detail
GET	/v1/referrals | /v1/referrals/{id}	Codes, campaigns, progress, rewards, authorized detail
POST	/v1/referrals/validate	Validate attribution before completion
GET/ PATCH	/v1/notifications | /preferences	History, mark read/all, five categories and preferences
POST/ DELETE	/v1/installations	Register/revoke push installation
GET/POST	/v1/support/tickets	List/create support tickets
GET/POST	/v1/support/tickets/{id}/messages	Ticket detail/messages
GET	/v1/accounts/me/deletion-preflight	Typed blockers and safe resolution destinations
POST	/v1/accounts/me/deletion-requests	Start protected deletion workflow

Method	Path	Purpose
GET	/v1/admin/dashboard	Dashboard metrics and drill-down metadata
GET/POST	/v1/admin/hosts/...	Host lists, review and lifecycle actions
GET/POST	/v1/admin/payments/...	Money operations and reconciliation
GET/POST	/v1/admin/refunds/...	Refund decisions and processing
GET/POST	/v1/admin/reports/...	Safety/support investigations
GET/POST	/v1/admin/settings/...	Versioned configuration governance
POST/GET	/v1/admin/exports	Audited asynchronous CSV exports

Appendix D - WebSocket event catalog

Direction	Event	Meaning
Client -> server	session.resync	Request missed/full durable state after reconnect
Client -> server	call.accept | call.reject	Host response with commandId and expectedVersion
Client -> server	call.cancel | call.end	Participant command, idempotent and authorized
Client -> server	video.request	Eligible customer request
Client -> server	video.respond | video.cancel	Host response or customer cancellation
Client -> server	presence.heartbeat	Host presence/liveness hint
Client -> server	call.telemetry	Bounded supplementary network/media evidence
Server -> client	call.ringing	Incoming/outgoing durable call intent summary
Server -> client	call.accepted | rejected | expired | cancelled	Ringing outcome
Server -> client	call.state	Full versioned call/media/timer/financial snapshot
Server -> client	video.eligible	Once-only server 180-second eligibility
Server -> client	video.requested | accepted | rejected | cancelled | expired	Mutual-consent state
Server -> client	call.reconnecting | recovered	Grace/recovery state
Server -> client	wallet.updated | low_balance	Versioned projection/warning
Server -> client	call.ending | ended | finalizing	Terminal/finalization progress
Server -> client	account.restricted | session.revoked	Immediate access/safety action

Envelope rule: Every command/event is versioned and includes event/command ID, call or aggregate ID, aggregate version, occurredAt, correlation ID, and a schema-validated payload.

Appendix E - Background jobs and durable events

Job	Purpose	Reliability rule
outbox.publish	Publish committed domain events	Short retry; inbox dedupe
otp.cleanup	Expire OTP challenges/rate keys	Scheduled; no auth impact on failure
media.scan/transcode	Validate and normalize uploaded audio/evidence	Idempotent by object hash/version
media.moderate/tag	Transcript moderation and mind tag derivation	Manual-review fallback
audio_status.expire	Remove expired status from public projection	Time-zone tested
presence.sweep	Mark stale hosts non-callable	Redis-derived, DB audit if needed
call.ring_timeout	Expire unanswered call	Durable deadline and expected version
call.reconcile	Merge Agora/client evidence	Dedupe provider event; state monotonic
call.billing_tick	Extend hold/post cumulative delta	Idempotent call+segment version
call.finalize	Close segments, charge, release hold, summary	Retriable; single checksum/version
payment.reconcile	Fetch missing/pending provider state	Rate-limited; one credit
promotion.release/reverse	Post or reverse eligible pack bonus	Campaign snapshot; separate promotional liability
refund.process/reconcile	Submit/fetch provider refund	One provider command
earnings.release	Move eligible pending to available	Hold-aware and ledger-backed
rank.calculate	Versioned rank and leaderboard run	Deterministic period/version
referral.qualify	Advance qualification/release reward	Source-event and fraud review
withdrawal.process/ reconcile	Submit/fetch payout	Locked funds; provider dedupe
notification.send	Send FCM/APNs and record result	Dedupe, TTL, invalid-token cleanup
callback_invitation.send	Deliver host callback notification	Prior-call eligibility, cooldown, preference and block checks
projection.rebuild	Rebuild dashboard/list projections	Checkpointed and replayable
retention.execute	Delete/anonymize expired eligible data	Legal-hold aware
account_deletion.execute	Apply approved anonymization/deletion plan	Blocker recheck; ledger/legal retention safe
export.generate	Create permissioned CSV artifact	Formula-safe, expiring, audited

Appendix F - Critical state transitions

Aggregate	Primary path	Non-negotiable rule
Call	intent_created -> ringing -> accepted -> connecting -> active_voice	Only server commands/provider evidence advance

Aggregate	Primary path	Non-negotiable rule
Video	locked -> eligible -> requested -> accepted -> transitioning -> active_video	180 seconds + customer request + host accept + media evidence
Recovery	active_* -> reconnecting -> active_previous | ending	Billing/eligibility paused while dual connection absent
Payment	order_created -> checkout_pending -> verification_pending -> captured/failed/reversed	Wallet credit only on verified captured
Promotion	eligible -> pending_payment -> released | rejected | reversed | expired	Server campaign snapshot; separate non-withdrawable ledger value
Refund	requested -> review -> approved/rejected -> processing -> completed/failed	Compensating ledger plus provider confirmation
Host application	draft -> submitted -> processing/review -> approved | corrections | rejected	Voice approval required; no selfie requirement
Withdrawal	draft -> submitted -> review -> approved -> processing -> completed/failed/held	Available balance locked; provider-confirmed completion
Referral	attributed -> progressing -> qualified -> held -> rewarded | rejected | expired	Campaign snapshot; ledger-backed reward
Account deletion	preflight_blocked | eligible -> requested -> accepted -> processing -> completed	Recheck blockers; retain only narrowly required records

Appendix G - Money and billing rules

- Persist all monetary values as integer paise and currency INR. Never use JavaScript number arithmetic for decimal rupee calculations without integer conversion and range checks.
- For a media mode with rate R paise per minute and cumulative connected duration D milliseconds, define one versioned rounding formula. A fair default is cumulativeCharge = ceil(R * D / 60,000); each billing run posts cumulativeCharge minus amountAlreadyPosted.
- The cumulative formula makes total charge independent of tick size. Processing one 60-second job or sixty 1-second jobs must produce the same final paise.
- Purchased customer wallet liability, promotional wallet liability, host pending payable, host available payable, platform revenue, payment clearing, payout clearing, refunds, and adjustments are distinct ledger accounts.
- Add-money bonus/cashback is calculated from a server campaign snapshot and posted separately from purchased credit. It is rupee-denominated promotional value, not coins or withdrawable cash; spend, expiry, refund and reversal ordering are explicit.
- Every business operation posts one balanced ledger transaction. Reversals and corrections are new compensating transactions linked to the original; original entries are never edited or deleted.

- Call start reserves configured minimum funds. Usage consumes/extends authorization in chunks. Unused reservation is released; it is not a charge.
- Ringing, one-sided connection, reconnect gaps, and video request/permission time are not billable. Video begins only after mutual consent and server/provider media evidence.
- Historical call charge and host earning use snapshots captured at intent/call creation. Later price or rank changes are prospective.

Example: At INR 6.00/minute, R = 600 paise. For 90,000 ms of eligible voice, charge = ceil(600 x 90,000 / 60,000) = 900 paise = INR 9.00. If 600 paise was already posted, the next delta is 300 paise.

Appendix H - Environment variable groups

Group	Examples	Rule
Runtime	NODE_ENV, APP_ENV, API_PORT, PUBLIC_BASE_URL, RELEASE, LOG_LEVEL	Validate; no secret except as documented
Database	DATABASE_URL, MIGRATION_DATABASE_URL, pool/timeouts	Secret store; separate migration role
Redis	REDIS_URL, prefix, queue/socket settings	TLS and environment prefix
Auth	JWT issuer/audience/signing keys, access/refresh TTL, encryption keys	Key Vault; rotation and key IDs
OTP	OTP_PROVIDER, credentials, sender/template IDs, fixed dev code	Dev code forbidden in production
Agora	AGORA_APP_ID, certificate, webhook secret, token TTL	App ID may be public; certificate is server-only
Razorpay	key ID, key secret, webhook secrets, account settings	Secret rotation supports old webhook retries
Payout	provider credentials, webhook secret, limits	Separate from payment gateway
Storage/media	account/container, CDN host, SAS/signing key, scanner/STT/moderation	Private containers and short URLs
Notifications	Firebase project/service identity, APNs configuration	Trusted server identity only
Observability	OTEL endpoint, Sentry DSN/auth, sampling, Azure connection	Scrub-before-send
App/deep links	minimum builds, store URLs, maintenance, allowed schemes/hosts, nonce signing	Public bootstrap signed/versioned; safe restore after login
Feature flags	video, audio status, mind matching, referrals, withdrawals, promotions, automation	Backend-owned and audited

Appendix I - Fastest safe execution plan

Milestone	Prompts	Deliverable	Gate
Milestone 0	Prompts 1-12	Repository, architecture, contracts, schema, auth/security base, providers, observability	Do not start live features before this gate
Milestone 1	Prompts 13-25	Customer/host identity, voice verification, self-intro, daily status, mind tags, discovery, safety	Frontend can begin real auth/profile/discovery integration
Milestone 2	Prompts 26-43	Pricing, ledger, top-up, call signaling/Agora, 180-second video, billing, refunds/reports	First complete paid-call staging journey
Milestone 3	Prompts 44-50	Earnings, ranks, payouts, referrals, host operations	Host monetization staging journey
Milestone 4	Prompts 51-56	Admin auth plus six modules	Operations team can safely review and act
Milestone 5	Prompts 57-68	Compliance engineering, security, tests, load, backup, integration, Azure, runbooks, release	Production go/no-go evidence

Codex working discipline

- One prompt equals one reviewed change set. Combine prompts only when they modify the same invariant and can still be tested independently.
- Never let two parallel agents edit the Prisma schema/migrations, shared API contracts, ledger, or call state machine at the same time.
- Safe parallel work after contracts exist: media provider adapter, notification templates, admin read projections, test fixtures, documentation, and IaC in separate files/branches.
- Require Codex to show changed files, migrations, commands, test output, contract changes, and security/privacy impact after every prompt.
- Integrate the mobile app progressively in preview/staging; do not wait until Prompt 68 to discover contract mismatches.

Appendix J - Official documentation baseline

Reference	Official URL
NestJS configuration	https://docs.nestjs.com/techniques/configuration
NestJS validation	https://docs.nestjs.com/techniques/validation
NestJS authentication	https://docs.nestjs.com/security/authentication
NestJS WebSocket gateways	https://docs.nestjs.com/websockets/gateways
NestJS queues	https://docs.nestjs.com/techniques/queues

Reference	Official URL
PostgreSQL transaction isolation	https://www.postgresql.org/docs/current/transaction-iso.html
PostgreSQL explicit locking	https://www.postgresql.org/docs/current/explicit-locking.html
Prisma Migrate	https://www.prisma.io/docs/orm/prisma-migrate
Prisma transactions	https://www.prisma.io/docs/orm/prisma-client/queries/transactions
Socket.IO Redis Streams adapter	https://socket.io/docs/v4/redis-streams-adapter/
BullMQ idempotent jobs	https://docs.bullmq.io/patterns/idempotent-jobs
Redis Streams	https://redis.io/docs/latest/develop/data-types/streams/
Agora RTC token server	https://docs.agora.io/en/realtime-media/rtc/build/authenticate-users/deploy-token-server
Agora channel event notifications	https://docs.agora.io/en/realtime-media/rtc/build/optimize-and-operate/receive-notifications
Razorpay webhooks	https://razorpay.com/docs/webhooks/
Razorpay webhook validation	https://razorpay.com/docs/webhooks/validate-test/
Firebase server environment	https://firebase.google.com/docs/cloud-messaging/server-environment
Firebase Admin send	https://firebase.google.com/docs/cloud-messaging/send/admin-sdk
Azure Container Apps architecture	https://learn.microsoft.com/en-us/azure/well-architected/service-guides/azure-container-apps
MeitY DPDP Rules 2025	https://www.meity.gov.in/documents/act-and-policies/digital-personal-data-protection-rules-2025-gDOxUjMtQWa
CERT-In Directions 2022 PDF	https://www.cert-in.org.in/PDF/CERT-In_Directions_70B_28.04.2022.pdf
MeitY IT Rules 2021	https://www.meity.gov.in/documents/act-and-policies/information-technology-intermediary-guidelines-and-digital-media-ethics-code-rules-2021-it-rules-2021-IjM5QjMtQWa
RBI PPI FAQs	https://www.rbi.org.in/commonman/english/scripts/FAQs.aspx?Id=2812

Dependency versions are intentionally not printed as permanent requirements. Resolve current stable mutually compatible versions during Prompt 1-3, lock them in the repository, record them in docs/dependency-matrix.md, and review upgrades deliberately.

Appendix K - Screenshot-derived changes and integration decisions

The capture contains both intended product states and visible mock/navigation limitations. The backend must implement the intended route contracts while contract tests expose, rather than preserve, stale or unreachable frontend states.

Capture	Observed state	Backend decision and required test
Screens 25-37	Several intended onboarding routes rendered the same Gender screen.	Use server-owned nextStep and per-route schemas; add adapter/E2E tests that fail when a stale screen renders for another step.
Screens 92-105	Host onboarding routes displayed Requires host session placeholders.	Issue the correct role/application session after auth and authorize resumable host drafts; never weaken guards to make routes render.
Screens 81-83	Outgoing/voice/video-request routes had no selected host.	Return typed missing_context/host_unavailable outcomes and restore only signed, authorized call context.
Screens 84/132	Video route refused to render without an active video session.	Keep this guard. Backend requires 180 seconds, customer request, host accept and dual video evidence.
Screens 85/133	Reconnect shows 30 seconds and paused billing.	Snapshot a 30-second v1 deadline; pause usage/video eligibility and reject stale restore after timeout.
Screens 86-89/134-137	Report and summary routes lacked call context or showed skeletons.	Provide signed recent-call context plus pending/finalized polling; empty/skeleton is not a final financial state.
Screens 66-67	Deletion is blocked by wallet balance but action copy says Open add money.	Backend returns blocker/resolution codes; do not encode misleading UI copy. Product/legal policy chooses refund, support or another approved resolution.
Screens 68-70	Pack bonuses and payment result are locally mocked.	Server owns pack campaign, bonus calculation, payment verification, promotional ledger posting and retry status.
Screens 71-74	Referral milestones are visible; a detail fixture is not found.	Return campaign snapshot, milestone timestamps, reward ledger status, authorized detail and typed not-found/retry results.
Screens 75-80	Support attachments and transaction truth are local mocks.	Upload/scan real attachments, preserve drafts/idempotency, and serve immutable server transaction/source references.
Screens 106-128	Dashboard and standalone leaderboard use different visible metrics.	Require explicit metric and calculation version: earnings leaderboard versus completed-calls leaderboard.
Screens 110-111	Notify user appears on host recents.	Create a rate-limited callback invitation notification only; no chat, auto-call, phone disclosure or block bypass.
Screens 114-122	UPI plus two bank methods are shown.	Enforce one UPI, max two tokenized bank accounts and one primary bank; mask, verify, cool down changes and audit use.
Screens 129-130	Host sees Balance verified and estimated minutes, not an exact wallet amount.	Return only a coarse verified-coverage estimate derived server-side; never expose customer balance or payment details.
Screens 138-151	Developer, component, token, layout, motion and animation tools are present.	Keep deterministic fixtures for preview/test only. Production builds and APIs must disable or omit them.

Appendix L - 151-screen UI-to-backend traceability register

Coverage baseline: 151 screenshots inspected: 134 public/auth/customer/host/call states and 17 internal fixture/QA states. Top and scroll captures remain separate rows so no visible page state disappears from implementation review.

#	Screen/state	Captured route	Backend ownership / required contract
1	public/ login_initial_after_clean_launch (top)	connecto://expo-development-client/?url=http://127.0.0.1:8081	OS/provider failure surface; backend is not authoritative. Auth endpoints must remain retry-safe.
2	public/age_confirmation (top)	connecto:///(auth)/age-confirmation	Public bootstrap plus auth entry; server enforces 18+ from DOB before account activation.
3	public/referral_code_entry (top)	connecto:///(auth)/referral-code	Validate optional referral code/campaign; reserve attribution once before onboarding completes.
4	public/blocked_account (top)	connecto:///(auth)/blocked-account	Account-status projection: blocked reason class, safe support path, and restricted session behavior.
5	public/suspended_account (top)	connecto:///(auth)/suspended-account	Account-status projection: suspension state, review status, safe support path, and session restriction.
6	public/underage_block (top)	connecto:///(auth)/underage-block	Permanent/ineligible age-gate outcome; legal/support remain reachable without ordinary account access.
7	public/mandatory_update (top)	connecto://mandatory-update	Public app-bootstrap config: platform minimum build, store URL, maintenance state, and signed/versioned cache data.
8	public/invalid_link (top)	connecto://invalid-link	Deep-link resolver returns typed invalid/expired/unauthorized outcome and safe root destination.
9	public/legal_terms (top)	connecto://legal/terms	Public locale/versioned Terms document with effective date and acceptance requirement metadata.
10	public/legal_terms (scroll1)	connecto://legal/terms	Public locale/versioned Terms document with effective date and acceptance requirement metadata.
11	public/legal_privacy (top)	connecto://legal/privacy	Public locale/versioned Privacy document with effective date and retention/contact metadata.
12	public/legal_privacy (scroll1)	connecto://legal/privacy	Public locale/versioned Privacy document with effective date and retention/contact metadata.
13	public/ legal_community_guidelines (top)	connecto://legal/community-guidelines	Public locale/versioned Community Guidelines document and policy acceptance metadata.
14	public/ legal_community_guidelines (scroll1)	connecto://legal/community-guidelines	Public locale/versioned Community Guidelines document and policy acceptance metadata.
15	public_links/referral_link (top)	connecto://referral/CONNECTO100	Deep-link resolver pre-fills validated referral attribution without applying a reward on open.
16	public_links/support_ticket_link (top)	connecto://link/support-ticket/ticket_1	Restore login, then authorize ticket ownership before returning support-ticket destination.
17	public_links/payment_result_link (top)	connecto://link/payment-result	Restore login and resolve server-owned payment state; callback alone never credits wallet.
18	public_links/notification_link (top)	connecto://link/notification/calls	Restore login and resolve an allowlisted notification destination with resource authorization.

#	Screen/state	Captured route	Backend ownership / required contract
19	public_links/call_invitation_link (top)	connecto://link/call-invitation/invitation_1	Resolve short-lived signed invitation, then revalidate account, block, host, call, and expiry state.
20	auth_flow/login_empty (top)	connecto:///(auth)/login	OTP request plus optional verified Google OIDC path if the adapter is enabled; enumeration-safe responses.
21	auth_flow/ login_filled_new_customer (filled)	connecto:///(auth)/login	OTP request binds normalized phone, device, referral-attribution context, cooldown, and purpose.
22	auth_flow/otp_entry (top)	connecto:///(auth)/otp	OTP challenge status/resend timing; no fixed code outside explicit development mode.
23	auth_flow/otp_filled (filled)	connecto:///(auth)/otp	OTP verification issues rotating session and server-owned account-status/nextStep result.
24	customer_onboarding/ profile_setup_empty (top)	connecto:///(customer-onboarding)/profile-setup	Customer onboarding draft: display name, DOB, immutable-after-setup gender, and avatar catalog selection.
25	customer_onboarding/ profile_setup_filled_host_applicant (filled)	connecto:///(customer-onboarding)/profile-setup	Onboarding draft plus role/application intent; backend returns deterministic nextStep.
26	customer_onboarding/ language_selection (top)	connecto:///(customer-onboarding)/language	Supported-language catalog plus primary/secondary selection with optimistic concurrency.
27	customer_onboarding/ language_selection_telugu (selected)	connecto:///(customer-onboarding)/language	Persist selected primary language and return versioned nextStep.
28	customer_onboarding/ match_tags (top)	connecto:///(customer-onboarding)/match-tags	Curated matching-tag catalog; store explicit safe selections separately from derived mind tags.
29	customer_onboarding/ match_tags_selected (selected)	connecto:///(customer-onboarding)/match-tags	Persist selected curated tags with catalog version and return nextStep.
30	customer_onboarding/ permissions_review (top)	connecto:///(customer-onboarding)/ permissions	Record product preference/education state; OS permissions remain device-owned and just-in-time.
31	customer_onboarding/ voice_verification_initial (top)	connecto:///(customer-onboarding)/voice-verification	Issue random-phrase voice challenge, expiry, upload intent, and allowed language/duration.
32	customer_onboarding/ voice_verification_recording (recording)	connecto:///(customer-onboarding)/voice-verification	Client recording state; backend owns challenge validity and upload authorization only.
33	customer_onboarding/ voice_verification_recorded (recorded)	connecto:///(customer-onboarding)/voice-verification	Complete immutable challenge upload and expose processing/review status.
34	customer_onboarding/ host_approval_pending (top)	connecto:///(customer-onboarding)/host-approval	Host-application status projection with safe reason/action and polling/deep-link updates.
35	customer_onboarding/ host_agreement (top)	connecto:///(customer-onboarding)/host-agreement	Versioned host agreement retrieval and explicit acceptance evidence.
36	customer_onboarding/ host_agreement_accepted (accepted)	connecto:///(customer-onboarding)/host-agreement	Idempotent agreement acceptance advances application nextStep when other checks pass.
37	host_flow/ post_application_host_dashboard (top)	connecto:///(host)/dashboard	Host dashboard/readiness projection; pending/rejected hosts cannot become callable.
38	internal/ developer_scenarios_before_seed (top)	connecto://developer-scenarios	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
39	customer/home (top)	connecto:///(customer)/home	Wallet projection, Featured Experts discovery, filters, audio fallback, and voice-only Walkie-Talkie intent.

#	Screen/state	Captured route	Backend ownership / required contract
40	customer/home (scroll1)	connecto:///(customer)/home	Wallet projection, Featured Experts discovery, filters, audio fallback, and voice-only Walkie-Talkie intent.
41	customer/filters (top)	connecto:///(customer)/filters	Server-supported language, availability, quality, rank/tag facets and result count metadata.
42	customer/calls (top)	connecto:///(customer)/calls	Cursor customer call history grouped client-side from UTC records plus privacy-safe online hints.
43	customer/calls (scroll1)	connecto:///(customer)/calls	Cursor customer call history grouped client-side from UTC records plus privacy-safe online hints.
44	customer/call_detail (top)	connecto:///(customer)/calls/call_1	Authoritative finalized/pending call detail, billing verification, wallet-after-call, rating, support, and report links.
45	customer/call_detail (scroll1)	connecto:///(customer)/calls/call_1	Authoritative finalized/pending call detail, billing verification, wallet-after-call, rating, support, and report links.
46	customer/profile (top)	connecto:///(customer)/profile	Customer profile summary with member date, languages, avatar, and authorized navigation counts.
47	customer/profile (scroll1)	connecto:///(customer)/profile	Customer profile summary with member date, languages, avatar, and authorized navigation counts.
48	customer/profile_edit (top)	connecto:///(customer)/profile-edit	Editable display name/avatar/languages; DOB and gender read-only after completed setup.
49	customer/profile_edit (scroll1)	connecto:///(customer)/profile-edit	Editable display name/avatar/languages; DOB and gender read-only after completed setup.
50	customer/profile_edit (scroll2)	connecto:///(customer)/profile-edit	Editable display name/avatar/languages; DOB and gender read-only after completed setup.
51	customer/settings (top)	connecto:///(customer)/settings	Account/legal/FAQ/logout/delete destinations; app version and motion preference are client-owned.
52	customer/settings (scroll1)	connecto:///(customer)/settings	Account/legal/FAQ/logout/delete destinations; app version and motion preference are client-owned.
53	customer/language_settings (top)	connecto:///(customer)/language-settings	Supported-language catalog and versioned customer preference update.
54	customer/language_settings (scroll1)	connecto:///(customer)/language-settings	Supported-language catalog and versioned customer preference update.
55	customer/motion_settings (top)	connecto:///(customer)/motion-settings	Client-only accessibility, motion, and vibration preference; no backend dependency in v1.
56	customer/motion_settings (scroll1)	connecto:///(customer)/motion-settings	Client-only accessibility, motion, and vibration preference; no backend dependency in v1.
57	customer/notifications (top)	connecto:///(customer)/notifications	Notification history, unread count, mark-one/mark-all, typed category, and allowlisted destination.
58	customer/notifications (scroll1)	connecto:///(customer)/notifications	Notification history, unread count, mark-one/mark-all, typed category, and allowlisted destination.
59	customer/ notification_preferences (top)	connecto:///(customer)/notification-preferences	Payments/Calls/Safety/Referrals/Support server preferences; OS permission remains separate.
60	customer/ notification_preferences (scroll1)	connecto:///(customer)/notification-preferences	Payments/Calls/Safety/Referrals/Support server preferences; OS permission remains separate.
61	customer/privacy_safety (top)	connecto:///(customer)/privacy-safety	Safety summary projection with block count, report count/status, policies, and safe destinations.
62	customer/privacy_safety (scroll1)	connecto:///(customer)/privacy-safety	Safety summary projection with block count, report count/status, policies, and safe destinations.

#	Screen/state	Captured route	Backend ownership / required contract
63	customer/blocked_users (top)	connecto:///(customer)/blocked-users	Authorized blocked-host list plus idempotent unblock; enforcement also applies to discovery/calls.
64	customer/report_history (top)	connecto:///(customer)/report-history	Cursor report history with safe status/category/target metadata; reporter/internal evidence stays private.
65	customer/report_history (scroll1)	connecto:///(customer)/report-history	Cursor report history with safe status/category/target metadata; reporter/internal evidence stays private.
66	customer/delete_account (top)	connecto:///(customer)/delete-account	Deletion preflight blockers, safe resolution actions, request/cancel status, and asynchronous execution.
67	customer/delete_account (scroll1)	connecto:///(customer)/delete-account	Deletion preflight blockers, safe resolution actions, request/cancel status, and asynchronous execution.
68	customer/add_money (top)	connecto:///(customer)/add-money	Wallet/transactions plus versioned pack catalog; bonus is server-calculated promotional ledger credit.
69	customer/payment_states (top)	connecto:///(customer)/payment-states	Server-owned payment result/status, amount, retry eligibility, and return destination.
70	customer/payment_states (scroll1)	connecto:///(customer)/payment-states	Server-owned payment result/status, amount, retry eligibility, and return destination.
71	customer/referrals (top)	connecto:///(customer)/referrals	Customer code/deep link, campaign copy, milestone progress, reward history, and ledger release status.
72	customer/referrals (scroll1)	connecto:///(customer)/referrals	Customer code/deep link, campaign copy, milestone progress, reward history, and ledger release status.
73	customer/referral_detail (top)	connecto:///(customer)/referrals/referral_1	Authorized referral detail with typed not-found/expired/retry states and no invitee private data.
74	customer/referral_detail (scroll1)	connecto:///(customer)/referrals/referral_1	Authorized referral detail with typed not-found/expired/retry states and no invitee private data.
75	customer/support (top)	connecto:///(customer)/support	Ticket create/list, categories, drafts, scanned attachments, source context, pagination, and retry-safe submit.
76	customer/support (scroll1)	connecto:///(customer)/support	Ticket create/list, categories, drafts, scanned attachments, source context, pagination, and retry-safe submit.
77	customer/support_ticket (top)	connecto:///(customer)/support-ticket/ticket_1	Authorized ticket detail, messages/status timeline, reference, category, timestamps, and source.
78	customer/support_ticket (scroll1)	connecto:///(customer)/support-ticket/ticket_1	Authorized ticket detail, messages/status timeline, reference, category, timestamps, and source.
79	customer/transaction_detail (top)	connecto:///(customer)/transaction/transaction_1	Immutable wallet transaction detail with source reference, linked call, status, and support action.
80	customer/transaction_detail (scroll1)	connecto:///(customer)/transaction/transaction_1	Immutable wallet transaction detail with source reference, linked call, status, and support action.
81	customer_calls/outgoing_call (top)	connecto://call/outgoing	Idempotent voice call intent, host claim, reservation, ring expiry, and no-host-selected typed state.
82	customer_calls/voice_call (top)	connecto://call/voice	Authoritative active-voice snapshot, connected timer, controls, price, wallet/reservation, and video progress.
83	customer_calls/video_request (top)	connecto://call/video-request	Exactly 180-second server eligibility plus customer request and host response state; voice remains active.
84	customer_calls/video_call (top)	connecto://call/video	Render only after mutual consent and dual video media evidence; otherwise typed return-to-voice state.
85	customer_calls/reconnecting (top)	connecto://call/reconnecting	Durable 30-second reconnect deadline, paused billing/eligibility, restore resync, or terminal timeout.

#	Screen/state	Captured route	Backend ownership / required contract
86	customer_calls/report_call (top)	connecto://call/report	Active/recent call-bound idempotent safety report with optional submit-and-end/block action.
87	customer_calls/report_call (scroll1)	connecto://call/report	Active/recent call-bound idempotent safety report with optional submit-and-end/block action.
88	customer_calls/call_summary (top)	connecto://call/summary	Pending/finalized summary polling with durations, charge, balance, rating, report, and support actions.
89	customer_calls/call_summary (scroll1)	connecto://call/summary	Pending/finalized summary polling with durations, charge, balance, rating, report, and support actions.
90	internal/ developer_scenarios_before_seed (top)	connecto://developer-scenarios	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
91	internal/ developer_scenarios_before_host_toggle (top)	connecto://developer-scenarios	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
92	host_onboarding/profile_setup (top)	connecto:///(host-onboarding)/host-profile-setup	Protected resumable host public/private profile draft and server-owned nextStep.
93	host_onboarding/profile_setup (scroll1)	connecto:///(host-onboarding)/host-profile-setup	Protected resumable host public/private profile draft and server-owned nextStep.
94	host_onboarding/photo (top)	connecto:///(host-onboarding)/photo	Moderated public profile media only; never selfie identity verification.
95	host_onboarding/photo (scroll1)	connecto:///(host-onboarding)/photo	Moderated public profile media only; never selfie identity verification.
96	host_onboarding/rules (top)	connecto:///(host-onboarding)/rules	Versioned host rules acknowledgement evidence.
97	host_onboarding/rules (scroll1)	connecto:///(host-onboarding)/rules	Versioned host rules acknowledgement evidence.
98	host_onboarding/verification (top)	connecto:///(host-onboarding)/verification	Random-phrase voice challenge and review status; payout KYC remains separate.
99	host_onboarding/verification (scroll1)	connecto:///(host-onboarding)/verification	Random-phrase voice challenge and review status; payout KYC remains separate.
100	host_onboarding/payout (top)	connecto:///(host-onboarding)/payout	Protected payout/KYC readiness and masked beneficiary setup.
101	host_onboarding/payout (scroll1)	connecto:///(host-onboarding)/payout	Protected payout/KYC readiness and masked beneficiary setup.
102	host_onboarding/agreement (top)	connecto:///(host-onboarding)/agreement	Versioned host agreement retrieval and idempotent acceptance.
103	host_onboarding/agreement (scroll1)	connecto:///(host-onboarding)/agreement	Versioned host agreement retrieval and idempotent acceptance.
104	host_onboarding/ application_states (top)	connecto:///(host-onboarding)/application-states	Submitted/processing/review/corrections/approved/rejected/appeal projection and safe next action.
105	host_onboarding/ application_states (scroll1)	connecto:///(host-onboarding)/application-states	Submitted/processing/review/corrections/approved/rejected/appeal projection and safe next action.
106	host/dashboard (top)	connecto:///(host)/dashboard	Host readiness/availability, incoming-call summary, earnings, rank, and explicit earnings leaderboard projection.
107	host/dashboard (scroll1)	connecto:///(host)/dashboard	Host readiness/availability, incoming-call summary, earnings, rank, and explicit earnings leaderboard projection.
108	host/calls (top)	connecto:///(host)/calls	Host call/earning history from the shared finalized call service.
109	host/calls (scroll1)	connecto:///(host)/calls	Host call/earning history from the shared finalized call service.

#	Screen/state	Captured route	Backend ownership / required contract
110	host/recent (top)	connecto:///(host)/recent	Cursor recents plus rate-limited preference-aware callback invitation; no chat/contact exposure.
111	host/recent (scroll1)	connecto:///(host)/recent	Cursor recents plus rate-limited preference-aware callback invitation; no chat/contact exposure.
112	host/earnings (top)	connecto:///(host)/earnings	Pending/available totals, period aggregates, immutable statement, and withdrawal history.
113	host/earnings (scroll1)	connecto:///(host)/earnings	Pending/available totals, period aggregates, immutable statement, and withdrawal history.
114	host/withdrawal (top)	connecto:///(host)/withdrawal	Available-balance validation, amount options, UPI/bank method, review state, history, and idempotent request.
115	host/withdrawal (scroll1)	connecto:///(host)/withdrawal	Available-balance validation, amount options, UPI/bank method, review state, history, and idempotent request.
116	host/profile (top)	connecto:///(host)/profile	Public host summary plus private masked payout/readiness/report projections under field-level authorization.
117	host/profile (scroll1)	connecto:///(host)/profile	Public host summary plus private masked payout/readiness/report projections under field-level authorization.
118	host/profile_edit (top)	connecto:///(host)/profile-edit	Moderated public profile update plus one UPI and up to two masked bank methods with safe replacement.
119	host/profile_edit (scroll1)	connecto:///(host)/profile-edit	Moderated public profile update plus one UPI and up to two masked bank methods with safe replacement.
120	host/profile_edit (scroll2)	connecto:///(host)/profile-edit	Moderated public profile update plus one UPI and up to two masked bank methods with safe replacement.
121	host/bank_account_add (top)	connecto:///(host)/bank-account-add	Tokenized beneficiary creation/verification, primary selection, cooling period, and masked response.
122	host/bank_account_add (scroll1)	connecto:///(host)/bank-account-add	Tokenized beneficiary creation/verification, primary selection, cooling period, and masked response.
123	host/referrals (top)	connecto:///(host)/referrals	Host code, invites/qualified/reward counters, campaign qualification, and earnings-ledger reward history.
124	host/referrals (scroll1)	connecto:///(host)/referrals	Host code, invites/qualified/reward counters, campaign qualification, and earnings-ledger reward history.
125	host/settings (top)	connecto:///(host)/settings	Account/legal/FAQ/logout/deactivation destinations; app version and motion preference are client-owned.
126	host/settings (scroll1)	connecto:///(host)/settings	Account/legal/FAQ/logout/deactivation destinations; app version and motion preference are client-owned.
127	host/leaderboard (top)	connecto:///(host)/leaderboard	Explicit completed-calls leaderboard metric, rank tier, cursor paging, pinned self, freshness, and eligibility.
128	host/leaderboard (scroll1)	connecto:///(host)/leaderboard	Explicit completed-calls leaderboard metric, rank tier, cursor paging, pinned self, freshness, and eligibility.
129	host_calls/incoming_call (top)	connecto://call/incoming	Signed ringing state with privacy-safe caller summary, rate snapshot, verified coverage estimate, accept/decline.
130	host_calls/voice_call (top)	connecto://call/voice	Authoritative host active-voice snapshot and controls from the shared call aggregate.
131	host_calls/video_request (top)	connecto://call/video-request	Host receives customer request only after server eligibility; explicit accept/reject and expiry.
132	host_calls/video_call (top)	connecto://call/video	Connected video snapshot only after consent/media evidence; otherwise return to voice.

#	Screen/state	Captured route	Backend ownership / required contract
133	host_calls/reconnecting (top)	connecto://call/reconnecting	Same durable 30-second reconnect policy and non-billable gap as the customer projection.
134	host_calls/report_call (top)	connecto://call/report	Host active/recent call-bound safety report with strict evidence privacy and idempotency.
135	host_calls/report_call (scroll1)	connecto://call/report	Host active/recent call-bound safety report with strict evidence privacy and idempotency.
136	host_calls/call_summary (top)	connecto://call/summary	Host pending/finalized summary with durations, earning, hold/status, report, and support actions.
137	host_calls/call_summary (scroll1)	connecto://call/summary	Host pending/finalized summary with durations, earning, hold/status, report, and support actions.
138	internal/developer_scenarios (top)	connecto://developer-scenarios	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
139	internal/developer_scenarios (scroll1)	connecto://developer-scenarios	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
140	internal/developer_scenarios (scroll2)	connecto://developer-scenarios	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
141	internal/component_gallery (top)	connecto://component-gallery	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
142	internal/component_gallery (scroll1)	connecto://component-gallery	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
143	internal/token_gallery (top)	connecto://token-gallery	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
144	internal/token_gallery (scroll1)	connecto://token-gallery	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
145	internal/layout_qa (top)	connecto://layout-qa	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
146	internal/layout_qa (scroll1)	connecto://layout-qa	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
147	internal/motion_lab (top)	connecto://motion-lab	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
148	internal/motion_lab (scroll1)	connecto://motion-lab	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
149	internal/animation_lab (top)	connecto://animation-lab	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
150	internal/animation_lab (scroll1)	connecto://animation-lab	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.
151	internal/account_status_redirect (top)	connecto://account-status	Non-production fixture only. Use deterministic local seed/fake state; production route returns disabled/not-found.

Appendix M - Screen-derived response and error contract

Surface	Required fields	Typed states
App bootstrap	minBuild, latestBuild, updateRequired, storeUrl, maintenance, legalVersions, features, supportedLanguages, configVersion	normal, update-required, maintenance, stale-cache

Surface	Required fields	Typed states
Auth/session outcome	accountId, roles, accountStatus, onboardingRole, nextStep, continuation, access/refresh token metadata	new, active, underage, blocked, suspended, deletion-pending, policy-reaccept
Onboarding step	role, step, draftVersion, completedSteps, fields, blockers, nextStep	empty, draft, processing, corrections, approved/rejected
Discovery page	items, facets, appliedFilters, totalEstimate, nextCursor, fallbackLanguage, algorithmVersion, freshness	results, fallback, empty, stale, restricted
Host card	publicHostId, displayName, ageBand/value if approved, city, languages, availability, rank, rate, audioProjection, newHost	callable, busy, offline, removed
Wallet	purchasedPaise, promotionalPaise, heldPaise, spendablePaise, projectionVersion	funded, empty, held, reconciliation-pending
Wallet pack	packId, basePaise, bonusPaise, badge, eligibility, campaignVersion, expiresAt	available, ineligible, expired, superseded
Payment	paymentId, orderId, basePaise, bonusPaise, status, creditedTransactionIds, retryAllowed, updatedAt	pending, verified, failed, cancelled, reversed, manual-review
Call state	callId, aggregateVersion, mode, participants, connectedMs, videoRemainingMs, reconnectDeadline, financialSnapshot, allowedCommands	ringing through terminal typed states
Call summary/detail	finalizationStatus, voice/video ms, rates, charge/earning, balance/hold, endReason, verification, rating, support/report links	pending, verified, adjusted, unavailable
Leaderboard	metric, period, language, calculationVersion, items, pinnedSelf, nextCursor, calculatedAt	ready, insufficient-data, under-review, stale
Payout method	methodId, type, maskedLabel, bank/UPI display metadata, primary, verification, coolingUntil, version	pending, verified, rejected, disabled
Withdrawal	withdrawalId, amountPaise, method, status, availableAfterPaise, requestedAt, review/provider references	submitted, review, processing, completed, failed, rejected, held
Referral	program, code, deepLink, campaignVersion, milestones, rewardPaise, rewardStatus, safe counters	progressing, qualified, held, released, rejected, expired, reversed
Notification	notificationId, category, title/body template result, readAt, createdAt, destinationType/id	unread, read, destination-unavailable
Support/report	caseId/reference, category, safe subject, source, status, timeline, attachment metadata, updatedAt	draft, open, waiting, review, resolved, dismissed
Deletion preflight	eligible, blockers[{code,message,action,destination}], requestStatus, recheckToken, policyVersion	blocked, eligible, requested, accepted, processing, completed
Problem response	type, title, status, code, safeDetail, requestId, fieldErrors, retryAfter, currentVersion	validation, auth, forbidden, conflict, not-found, rate-limit, provider-pending

Contract rule: Every response uses stable opaque IDs, UTC timestamps, integer paise, aggregate/resource versioning, safe reason codes and explicit authorization. Display copy, local date formatting, motion and animation remain frontend responsibilities unless versioned content is served by the backend.

Appendix N - Final production go/no-go checklist

- Clean checkout installs, migrates, seeds, starts API/worker, and passes all format, lint, type, unit, integration, contract, E2E, security, and migration checks.
- All 151 traceability rows are implemented or explicitly classified as client-only/non-production, and every production screen's loading, empty, pending, error and retry states pass adapter tests.
- No direct-video endpoint/event exists. Server proves 180,000 ms dual-connected voice, mutual consent, and video media evidence before video billing.
- Purchased wallet value, promotional wallet value, payment, call billing, refund, host earning, referral reward, withdrawal, and payout reconcile to balanced immutable ledger entries.
- Reconnect expires from a durable 30-second v1 deadline, bills no disconnected time, adds no video-eligibility time, and rejects stale restoration.
- Walkie-Talkie creates only a one-to-one voice intent; callback invitations create only rate-limited notifications; neither opens chat, rooms, groups or direct video.
- Razorpay and payout webhook retries/out-of-order events cannot duplicate money. Agora/provider event retries cannot duplicate segments or reopen calls.
- Host voice verification, self-introduction, daily audio status, and mind text are separate workflows with correct privacy, moderation, and fallback.
- Blocked/restricted/suspended users cannot bypass enforcement through REST, sockets, RTC token, presence, discovery, or admin client manipulation.
- Admin has MFA, least privilege, step-up/dual control where configured, and append-only audit for every sensitive action.
- Production uses live/sandbox providers exactly as approved, developer fakes are disabled, secrets are managed, and logs/traces/analytics are redacted.
- Load target, backups/PITR, restore drill, monitoring, P0 alerts, and incident runbooks have evidence and owners.
- Legal, payments, payout/KYC, tax, privacy, recording/moderation, refund, grievance, store, and provider go-live reviews are signed off by the appropriate professionals/owners.
- Mobile and admin critical journeys pass against staging real adapters and provider sandboxes with no hidden mock fallback.
- BACKEND_RELEASE_REPORT.md says GO with evidence; otherwise launch remains NO-GO with a bounded remediation list.